UNIVERSIDAD DE ANTIOQUIA

Ajuste de parámetros sobre wav2vec 2.0 para la clasificación de la enfermedad de Alzheimer mediante señales de voz en condiciones de datos limitados

Santiago Alvarez Zuluaga

Trabajo de grado presentado para optar al título de Ingeniero de Telecomunicaciones

Modalidad de Práctica

Trabajo de Grado

Asesor

Cristian David Rios Urrego, Magíster (MSc) en Ingeniería de Telecomunicaciones

Universidad de Antioquia

Facultad de Ingeniería

Ingeniería de Telecomunicaciones

Medellín, Antioquia, Colombia

2026

Cita	Alvarez Zuluaga[1]
Referencia Estilo IEEE (2020)	[1] S. Alvarez Zuluaga , “Ajuste de parámetros sobre wav2vec 2.0 para la clasificación de la enfermedad de Alzheimer mediante señales de voz en condiciones de datos limitados”, Trabajo de grado profesional, Ingeniería de Telecomunicaciones, Universidad de Antioquia, Medellín, Antioquia, Colombia, 2026.

creative commons BY NC SA

Grupo de Investigación. Telecomunicaciones Aplicadas (GITA)

Linea de reconocimiento de patrones

Centro de Documentación Ingeniería (CENDOI)

Repositorio Institucional: http://bibliotecadigital.udea.edu.co

Universidad de Antioquia - www.udea.edu.co

El contenido de esta obra corresponde al derecho de expresión de los autores y no compromete el pensamiento institucional de la Universidad de Antioquia ni desata su responsabilidad frente a terceros. Los autores asumen la responsabilidad por los derechos de autor y conexos.

UNIVERSIDAD DE ANTIOQUIA

TABLA DE CONTENIDO

RESUMEN..... 10

ABSTRACT..... 11

I. INTRODUCCIÓN..... 12

A. *Planteamiento del problema*..... 13

B. Estado del arte..... 14

C. Justificación..... 16

II. OBJETIVOS..... 17

A. Objetivo general..... 17

B. Objetivos específicos..... 17

III. MARCO TEÓRICO..... 18

A. wav2vec 2.0..... 18

1) Pre entrenamiento de wav2vec 2.0:..... 18

2)Feature encoder:..... 22

3) Encoder transformer:..... 23

a) Atención multi-header..... 24

b) Suma residual..... 24

c) LayerNorm..... 24

d) Feed-forward..... 25

3) Etapa de clasificación:..... 25

4) Interpretabilidad por capas de representación auto-supervisadas:..... 26

B. Parameter-Efficient Fine-Tuning(PEFT)..... 27

1) Adaptadores (Adapters):..... 29

2). Low-Rank Adaptation (LoRA):..... 31

IV. BASE DE DATOS 34

V. METODOLOGÍA 35

A. Preprocesamiento: 36

1) Remuestreo 36

2) Enventanado 36

C. Elección del modelo de wav2vec 2.0 37

D. Estrategias de ajuste 39

1) Descongelamiento parcial 40

4) Adapters de tipo bottleneck 41

5) LoRA. 42

E. Etapa de clasificación 43

V. ANÁLISIS DE RESULTADOS 44

A. Referencia: Segmentos originales de ADReSS 44

B. Enventanado 44

C. Tamaño de ventana de 6 segundos. 45

C. Resultados con ventana de 7.5 segundos 50

VI. DISCUSIÓN 56

+VI. CONCLUSIONES Y RECOMENDACIONES 57

REFERENCIAS 59

ANEXOS 61

UNIVERSIDAD DE ANTIOQUIA

1803

LISTA DE TABLAS

TABLA I: RESULTADO DE EXPERIMENTOS PARA VENTANAS DE 7.5 SEGUNDOS....36
TABLA II: DESEMPEÑO SOBRE LOS SEGMENTOS ORIGINALES DE ADReSS.....41
TABLA III: DESEMPEÑO DEL BASELINE POR TAMAÑO DE VENTANA.....42
TABLA IV: RESULTADO DE EXPERIMENTOS PARA VENTANAS DE 6 SEGUNDOS....43
TABLA V: PARAMETROS ENTRENABLES POR CONFIGURACIÓN EN VENTANAS DE 6 SEGUNDOS.....48
TABLA VI: RESULTADO DE EXPERIMENTOS PARA VENTANAS DE 7.5 SEGUNDOS. 49
TABLA VII: PARAMETROS ENTRENABLES POR CONFIGURACIÓN EN VENTANAS DE 6 SEGUNDOS.....54

UNIVERSIDAD DE ANTIOQUIA

1803

LISTA DE FIGURAS

Fig. 1: Flujo del preentrenamiento de wav2vec 2.0.....18
Fig. 2: Arquitectura de wav2vec 2.0 para clasificación: (a) feature encoder, (b) encoder Transformer y (c) etapa de clasificación.....21
Fig. 3. Estructura de una capa del encoder Transformer de wav2vec 2.0.....22
Fig. 4: Taxonomía de métodos PEFT, con los adapters y LoRA resaltados. Adaptada de Lialin et al. [20].....27
Fig. 5: Estructura interna de un adapter bottleneck.....28
Fig. 6: Factorización de rango de la actualización de pesos ΔW en las matrices de bajo rango B y A..... 30
Fig. 7: Metodologia implementada.....33
Fig. 8: Adaptador Bottleneck en configuración Pfeiffer.....39
Fig. 9: Adaptador Bottleneck en configuración Houlsby.....39
Fig. 10: Exactitud obtenida por cada configuración.....45
Fig. 11: Curvas ROC y diagramas de distribución para el mejor modelo de 6 seg.....46
Fig. 12. Exactitud de modelos para ventanas de 7.5 segundos.....51
Fig. 13: Curvas ROC y diagramas de distribución para el mejor modelo de 7.5 segundos.....52

UNIVERSIDAD DE ANTIOQUIA

1803

SIGLAS, ACRÓNIMOS Y ABREVIATURAS

AD	Enfermedad de Alzheimer (Alzheimer's Disease)
Adam	Estimación adaptativa de momentos (Adaptive Moment Estimation)
AdamW	Adam con weight decay desacoplado (Adam with decoupled Weight decay)
ADReSS	Alzheimer's Dementia Recognition through Spontaneous Speech
ASR	Reconocimiento automático del habla (Automatic Speech Recognition)
AUC	Área bajo la curva ROC (Area Under the Curve)
BDAE	Boston Diagnostic Aphasia Examination
CNN	Red neuronal convolucional (Convolutional Neural Network)
ComParE	Conjunto de características del Computational Paralinguistics Challenge

CTC	Connectionist Temporal Classification
CTR	Controles sanos
DANE	Departamento Administrativo Nacional de Estadística
eGeMAPS	extended Geneva Minimalistic Acoustic Parameter Set
Esc.	Escenario
FF	Capa lineal de proyección (Feed-Forward)
GELU	Unidad lineal de error gaussiano (Gaussian Error Linear Unit)
h	Horas
Ho	Adaptador tipo Houlsby
Hz	Hercio
k	Proyección de clave (key) del mecanismo de atención
kHz	Kilohercio

LayerNorm	Normalización de capa (Layer Normalization)
LCR	Líquido cefalorraquídeo
LoRA	Adaptación de bajo rango (Low-Rank Adaptation)
M	Millones
o	Proyección de salida (output) del mecanismo de atención
PEFT	Ajuste fino eficiente en parámetros (Parameter-Efficient Fine-Tuning)
PET	Tomografía por emisión de positrones (Positron Emission Tomography)
Pf	Adaptador tipo Pfeiffer
PSEN1	Presenilina 1
PWCCA	Análisis de correlación canónica ponderado por proyección (Projection-Weighted Canonical Correlation Analysis)
q	Proyección de consulta (query) del mecanismo de atención

ReLU	Unidad lineal rectificada (Rectified Linear Unit)
ROC	Característica operativa del receptor (Receiver Operating Characteristic)
s	Segundos
ULMFiT	Universal Language Model Fine-tuning
v	Proyección de valor (value) del mecanismo de atención
XLSR	Representaciones de habla interlingüísticas (Cross-Lingual Speech Representations)

UNIVERSIDAD DE ANTIOQUIA

1803

RESUMEN

La enfermedad de Alzheimer (AD) altera el habla desde etapas tempranas, lo que convierte a la voz en un biomarcador no invasivo y de bajo costo para su detección. Los modelos auto-supervisados como wav2vec 2.0 ofrecen representaciones acústicas robustas, pero su ajuste completo tiende a sobreajustar cuando los datos etiquetados son escasos. Este trabajo evalúa estrategias de ajuste fino de parámetros (PEFT) sobre wav2vec 2.0 para clasificar pacientes con AD y controles sanos a partir de habla espontánea del corpus ADReSS 2020. Se compararon tres estrategias de ajuste diferentes, descongelamiento parcial, los adaptadores bottleneck (Houlsby y Pfeiffer) y LoRA, bajo tres esquemas de ubicación: estándar, progresivo e inteligente. Un barrido previo mostró que las ventanas de hasta 4 s llevan al colapso del clasificador, por lo que los experimentos se realizaron con ventanas de 6 y 7.5 s. Con 6 s, el adaptador Houlsby ubicado en el bloque semántico alcanzó una exactitud de 83.3 % y un AUC de 0.879, frente a 66.7 % y 0.705 del modelo congelado. Con 7.5 s, LoRA inteligente obtuvo 77.1 % de exactitud (AUC de 0.818), entrenando solo el 0.026 % de los parámetros del modelo. Las configuraciones estándar no superaron el *baseline*, y en LoRA la adaptación de las matrices de valor y salida fue la más efectiva en ambas ventanas. Los resultados indican que ubicar la adaptación de forma informada importa más que el número de parámetros entrenados.

Palabras clave — **Enfermedad de Alzheimer, wav2vec 2.0, ajuste fino de parámetros, adaptadores, LoRA.**

UNIVERSIDAD DE ANTIOQUIA

1803

ABSTRACT

Alzheimer's disease (AD) affects the way people speak from the early stages of the disease. Therefore, voice can be used as a low-cost and non-invasive biomarker for detecting the disease. Self-supervised models such as wav2vec 2.0 learn strong acoustic representations; however, fine-tuning the entire model with scarce labeled data may lead to overfitting. This work assesses fine-tuning strategies for wav2vec 2.0 to classify patients with AD against healthy controls using spontaneous speech from the ADReSS 2020 corpus. Three different fine-tuning strategies were compared: partial unfreezing, bottleneck adapters (Houlsby and Pfeiffer), and LoRA, under three adaptation-location schemes: Standard, Progressive, and Smart. A preliminary sweep showed that windows shorter than 4 s led to classifier collapse; therefore, the experiments were conducted using 6- and 7.5-second windows. For 6-second windows, the Houlsby adapter located in the semantic block achieved an accuracy of 83.3% and an AUC of 0.873, compared with 66.7% accuracy and an AUC of 0.705 for the original frozen model. For 7.5-second windows, Smart LoRA achieved an accuracy of 77.1% (AUC = 0.818) while training only 0.026% of the model's parameters. The Standard configuration did not outperform the baseline. Furthermore, adaptation of the value and output matrices in LoRA was the most effective across both window lengths. These results suggest that strategically locating the adaptation may be more important than the number of trainable parameters.

Keywords **— Alzheimer's disease, wav2vec, parameter fine-tuning, adapters, LoRA**

I. INTRODUCCIÓN

A nivel mundial, la demencia ha sido reconocida por la comunidad internacional como una prioridad creciente de salud pública, dada su magnitud y su impacto en los sistemas de salud, las familias y los cuidadores [1]. Colombia no es ajena a esta tendencia: entre 2015 y 2024, las muertes asociadas a enfermedades mentales y de la conducta (entre ellas el Alzheimer) se multiplicaron casi cinco veces, y solo en 2024 se registraron 4.320 muertes atribuidas específicamente a esta enfermedad, según cifras del DANE [2]. Sin embargo, a pesar de esta carga creciente, estudios recientes señalan que la demencia continúa siendo un problema de salud pública invisibilizado en el país, sin que existan a la fecha políticas públicas específicas que respondan a su magnitud [3]. Esta brecha resulta particularmente paradójica si se considera que Colombia alberga uno de los hallazgos genéticos más relevantes a nivel mundial para la investigación de la enfermedad de Alzheimer (AD) de inicio temprano: la mutación PSEN1 E280A, conocida como «gen paisa», presente en una familia extendida de la región antioqueña y estudiada durante más de cuatro décadas como modelo único para comprender la progresión clínica de la enfermedad años antes de la aparición de los síntomas [4].

El Alzheimer es una enfermedad neurológica progresiva que, más allá del deterioro de la memoria, compromete funciones cognitivas asociadas al lenguaje, las dificultades comunicativas se cuentan entre los grupos de síntomas que con mayor frecuencia acompañan la enfermedad [5] y se ven reflejadas directamente en el habla, cuya producción exige la coordinación de procesos cognitivos, lingüísticos y motores, a medida que la enfermedad avanza reduciendo la fluidez, precisión y coherencia del discurso [6]. El diagnóstico del Alzheimer se ha apoyado tradicionalmente en la evaluación clínica y la neuroimagen estructural, complementadas en las últimas dos décadas por pruebas de biomarcadores en líquido cefalorraquídeo (LCR) y tomografía por emisión de positrones (PET); sin embargo, estas técnicas presentan baja sensibilidad en etapas tempranas y un acceso limitado debido a su costo elevado y su naturaleza invasiva [6]. Teniendo en cuenta este panorama, resulta urgente explorar alternativas de detección que superen las limitaciones de los métodos diagnósticos tradicionales. Muchas evaluaciones neurológicas, aunque clínicamente valiosas, no están diseñadas para ser realizadas

de forma masiva y constante atendiendo a la magnitud que el problema exige, retrasando la intervención terapéutica y empeorando el pronóstico de los pacientes [5]. Frente a estas barreras, los biomarcadores han cobrado relevancia como una estrategia complementaria capaz de ofrecer diagnósticos más accesibles y económicos [7]. En este contexto, el análisis de habla constituye un procedimiento simple, económico y no invasivo, con potencial para la detección del Alzheimer incluso en etapas preclínicas [5]. No obstante, el desarrollo de sistemas automáticos de clasificación basados en habla conlleva una limitación estructural reconocida en la literatura: el tamaño reducido de las muestras disponibles que responde a una dificultad en la recolección de datos de habla en poblaciones con deterioro cognitivo [5].

Buscando mitigar este problema los modelos de aprendizaje autosupervisado han revolucionado la manera en que se generan representaciones de habla, permitiendo capturar estructuras acústicas y lingüísticas ricas a partir de grandes volúmenes de audio sin necesidad de transcripciones o etiquetas manuales [8]. Sin embargo, este paradigma no elimina la dependencia de datos: el pre entrenamiento se apoya en corpus masivos no etiquetados, y el ajuste posterior a una tarea específica (el fine-tuning) asume también la disponibilidad de conjuntos de datos etiquetados suficientemente amplios para ajustar el gran número de parámetros que un modelo de este tamaño requiere.

A. Planteamiento del problema

Suponer el acceso a un gran volumen de datos etiquetados representa un gran problema al trasladar estos métodos al dominio clínico de las enfermedades neurodegenerativas, las bases de datos de habla con etiquetas de Alzheimer disponibles públicamente son, por naturaleza, reducidos. La base de datos *Alzheimer's Dementia Recognition through Spontaneous Speech* ADReSS reúne apenas 156 hablantes [8], esta cantidad de muestras es órdenes de magnitud menores que los corpus empleados para el pre entrenamiento de wav2vec 2.0.

Frente a esta limitación, han surgido estrategias de ajuste eficiente de parámetros, conocidas en inglés como Parameter-Efficient Fine-Tuning (PEFT) diseñadas justamente para adaptar modelos

de gran tamaño con pocos datos; sin embargo, estas estrategías se han explorado poco para la clasificación de enfermedades neurodegenerativas como el Alzheimer a partir de wav2vec u otros modelos. Es en esta ausencia donde se sitúa el aporte metodológico central del presente trabajo. ¿Cómo construir un clasificador robusto, capaz de distinguir entre hablantes con Alzheimer y controles sanos, en condiciones de datos limitados?

B. Estado del arte

El desafío ADReSS [9] estableció un punto de referencia común para la detección automática del Alzheimer a partir del habla: 156 hablantes balanceados por clase, edad y género, grabaciones con reducción de ruido y normalización de volumen, y una partición fija de entrenamiento y prueba. Su línea base acústica, construida con descriptores diseñados manualmente, alcanzó una exactitud de 62,5%, lo que evidenció que las características acústicas tradicionales capturan parcialmente la enfermedad. Haider et al. [10] evaluaron conjuntos estandarizados como eGeMAPS y ComParE en habla espontánea de pacientes con Alzheimer y encontraron que estos capturan parte de los cambios vocales asociados a la enfermedad, con una exactitud de 71,3 % para eGeMAPS, el mejor conjunto individual, y de 78,7 % al fusionar los distintos conjuntos. Estos resultados, sin embargo, se obtuvieron con un protocolo de evaluación propio, sobre la partición común de ADReSS, los mismos conjuntos mostraron un desempeño inestable y dependiente del clasificador [9]. Adicionalmente su alcance queda restringido a los descriptores definidos a priori a partir del conocimiento de un comité de expertos. Esta restricción motivó el uso del aprendizaje profundo, que busca aprender la representación directamente de la señal, aunque a costa de enfrentar el principal obstáculo de estos modelos: el tamaño reducido de los corpus clínicos. Rios-Urrego et al. [11] abordaron este obstáculo en la detección de la enfermedad de Parkinson entrenando redes convolucionales con habla de pacientes en un idioma y usaron los pesos aprendidos como punto de partida para otro idioma, congelando parte de sus capas y reentrenando las demás. De esta forma mostraron que reutilizar el conocimiento previamente aprendido en una red es una vía para compensar la escasez de datos. Sin embargo, el modelo original se entrenó con habla de pacientes etiquetada. La transferencia de los pesos trasladaba la dependencia de etiquetas clínicas de un corpus a otro sin eliminarla. Por lo tanto, se

requería aprender representaciones del habla sin etiquetas, de modo que los datos clínicos etiquetados solo fueran necesarios en la etapa final de adaptación.

wav2vec 2.0 [8] eliminó la dependencia de etiquetas en la etapa de preentrenamiento. Con apenas diez minutos de datos etiquetados, el modelo alcanzó un reconocimiento de habla competitivo. No obstante, fue diseñado y evaluado para reconocimiento de habla, y no es claro qué información aporta cada una de sus capas a otras tareas. Pasad et al. [12] propusieron un análisis de interpretabilidad observando capa por capa el contenido acústico, fonético y léxico del modelo. Encontraron una trayectoria similar a la de un autoencoder: las capas intermedias concentran la mayor información fonética, las finales vuelven a parecerse a la entrada y el fine-tuning modifica principalmente las capas superiores. Su análisis, sin embargo, se limitó a habla sana.

A pesar de contar ahora con una herramienta como wav2vec 2.0 que puede ayudar a evaluar enfermedades neurodegenerativas a partir de la voz, la adaptación a pequeñas bases de datos de habla patologica enfrenta un riesgo documentado. Berisha et al. [13] analizaron estudios publicados de clasificación clínica a partir del habla y mostraron que la exactitud reportada disminuye a medida que aumenta el tamaño de la muestra. Si los datos provienen de la misma distribución y describen el mismo fenómeno, un modelo confiable deberia mantener o mejorar su desempeño al aumentar la cantidad de datos. . Xiong et al. [14] mostraron que modificar los pesos de un gran modelo autosupervisado con un corpus clínico de tamaño limitado conduce a sobreajuste. Al ajustar el Transformer de wav2vec 2.0, la pérdida sobre hablantes no vistos aumentó a lo largo del entrenamiento, pese al uso de regularización. Los autores lo atribuyen al desequilibrio entre la gran capacidad del modelo y la escasez de datos de la tarea: el modelo termina capturando patrones de los datos de entrenamiento ajenos a la patología en lugar de las características relevantes, y generaliza mal. Esto sugiere que, con pocos datos, adaptar todo el modelo es arriesgado y que conviene entrenar solo una fracción de sus parámetros, una idea desarrollada originalmente en el procesamiento de lenguaje natural.

Algunas estrategias de adaptación permiten modificar el modelo sin perder por completo la información de las representaciones originales obtenidas en el preentrenamiento, ULMFiT [15] introdujo el descongelamiento gradual de capas, de las superiores a las inferiores, con tasas de

aprendizaje distintas por capa. Con 100 ejemplos etiquetados igualó a modelos entrenados desde cero con cien veces más datos. Por otro lado, Houlsby et al. [16] propusieron adaptadores tipo *bottleneck*: módulos entrenables con conexiones residuales insertados en cada capa. Los autores lograron un desempeño 0,4% por debajo de un fine-tuning completo pero ajustando solo 3,6 % de parámetros por tarea. Otro método propuesto en [17] es LoRA, esta estretegia mantiene fijas las matrices de pesos W y aprende una actualización de bajo rango, W' = W + BA, que se integra a los pesos sin añadir latencia, lo cual permitió reducir miles de veces los parámetros entrenables con un desempeño comparable en aplicaciones de texto. Sin embargo, Chen et al. [18] las trasladaron a modelos de habla autosupervisados y alcanzaron un desempeño comparable al *fine-tuning* completo reduciendo en más de un 90 % los parámetros entrenables, en tareas de reconocimiento del habla, clasificación e identificación de hablantes sobre habla de personas sanas.

Purohit et al. [18] evaluaron diferentes estrategias de adaptación en la detección de la enfermedad de Parkinson: compararon la selección de capas congeladas, el fine-tuning completo y LoRA sobre modelos de habla. Seleccionar la décima capa congelada de wav2vec 2.0 (83,5%) superó al fine-tuning completo (80,9%), y LoRA aplicado a Whisper alcanzó 85,0%, por encima de su contraparte ajustada por completo (83,5%). En conclusión, ninguna de estas estrategias se ha evaluado sobre wav2vec 2.0 para la clasificación de Alzheimer frente a controles sanos, una enfermedad cuya huella en el habla es principalmente cognitiva y lingüística.

C. Justificación

Frente a la escasez de datos en el fenómeno de interés. este trabajo propone aplicar y comparar diferentes estrategias de adaptación sobre wav2vec 2.0 para la tarea de clasificación binaria entre hablantes con Alzheimer y controles sanos: adaptadores tipo bottleneck, que insertan módulos secuenciales de baja dimensionalidad entre las capas del modelo; LoRA (Low Rank Adaptation), que introduce una corrección de bajo rango sobre las matrices de pesos existentes sin modificarlas directamente [17]; y el descongelamiento parcial de las últimas n capas del modelo, como forma más directa de ajuste. Estas tres estrategias serán evaluadas frente

a un mismo punto de referencia : wav2vec 2.0 sin ningún ajuste de sus pesos. A partir de este diseño experimental, la presente tesis plantea como hipótesis central que el ajuste del modelo, mediante cualquiera de las estrategias mencionadas, permite obtener un desempeño de clasificación superior al del modelo congelado, incluso en condiciones de datos limitados.

II. OBJETIVOS

A. Objetivo general

Desarrollar una estrategia de ajuste de parámetros sobre modelos fundacionales auto-supervisados, como wav2vec 2.0, para la clasificación de pacientes con enfermedad de Alzheimer (AD) frente a controles sanos (CTR) mediante señales de voz en condiciones de datos limitados.

B. Objetivos específicos

- Establecer una línea base de clasificación de pacientes con enfermedad de Alzheimer y sujetos de control mediante el uso de representaciones extraídas por wav2vec 2.0 sin ajuste de parámetros.
- Identificar estrategias de ajuste de parámetros aplicables a la arquitectura de wav2vec 2.0 para tareas de clasificación binaria en escenarios con datos limitados.
- Implementar diferentes estrategias de ajuste de parámetros sobre el modelo fundacional wav2vec 2.0, con el fin de adaptarlo a la tarea de clasificación binariaentre pacientes con la enfermedad de Alzheimer y sujetos de control.
- Evaluar y comparar el desempeño de clasificación, el tiempo de entrenamiento, el tiempo de inferencia y el número de parámetros entrenables entre el enfoque del modelo base y las estrategias de ajuste implementadas.

III. MARCO TEÓRICO

A. wav2vec 2.0

wav2vec 2.0 [8] es un modelo de aprendizaje autosupervisado que aprende representaciones de habla directamente del audio crudo, sin necesidad de transcripciones. Para comprender su arquitectura es importante distinguir dos etapas: el preentrenamiento autosupervisado y el uso posterior del modelo preentrenado. Durante el preentrenamiento, el modelo aprende sus representaciones mediante una arquitectura compuesta por un codificador convolucional (*feature encoder*), una red Transformer (*encoder transformer)* que construye representaciones contextuales y un módulo de cuantización con dos *codebooks*, cuyas salidas se emplean en dos funciones de pérdida: una contrastiva y otra de diversidad. La Fig. 1 resume esta arquitectura y el flujo de información durante el preentrenamiento. En la segunda etapa, en cambio, se parte del modelo ya preentrenado y solo se conservan el *feature encoder* y el Transformer; el cuantizador, los *codebooks* y las pérdidas de preentrenamiento se descartan, pues su única función es proporcionar la señal de aprendizaje durante el preentrenamiento.

1) pre entrenamiento de wav2vec 2.0:

El *feature encoder* transforma la señal cruda en una secuencia de vectores latentes continuos $z_1, \ldots . z_T$, uno por cada fotograma de 25 ms. Esta secuencia z alimenta en paralelo dos ramas independientes durante el *forward*. En la primera, antes de ingresar al *Transformer*, se enmascaran tramos contiguos: se muestrea el 65% de los pasos como índices de inicio, y desde cada uno se cubren 10 pasos consecutivos con un único vector entrenable compartido (el *mask embedding*). Por el solapamiento entre tramos, el resultado es que cerca del 49% de la secuencia queda enmascarada. Esta estrategia de bloques (en vez de fotogramas aislados) sigue una lógica similar a *SpecAugment [19]*: los fotogramas vecinos están altamente correlacionados, de modo que enmascarar uno solo permitiría reconstruirlo por simple interpolación, mientras que cubrir un tramo completo obliga al modelo a recurrir a contexto de largo alcance.

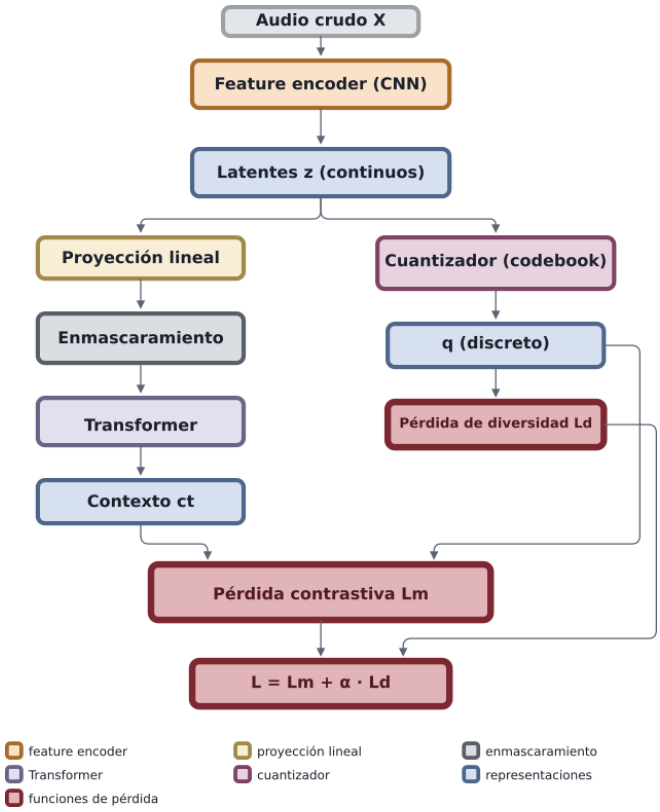


Fig. 1: Flujo del preentrenamiento de wav2vec 2.0

La secuencia z parcialmente enmascarada entra al Transformer, que produce una representación contextualizada c_t para cada posición mediante auto-atención: cada posición, actuando como *query* Q, calcula su similitud contra las *keys* K de todas las posiciones mediante el producto punto $Q \cdot K$; luego se normaliza esos puntajes con una función *softmax*, y construye su salida como una suma ponderada de los *values* Vde las posiciones a las que atiende. En las posiciones enmascaradas, dado que el *mask embedding* no contiene información real del fotograma original, c_t se reconstruye enteramente a partir del contexto (las posiciones vecinas no enmascaradas) y no de su propia entrada.

En la segunda rama, la misma secuencia *z* alimenta un cuantizador que discretiza cada vector en una representación de un conjunto finito de unidades aprendidas. Para ello emplea $G = 2$ *codebooks* cada uno con V=320 entradas representadas como una matriz $E_g \in \mathbb{R}^{V \times d/G}$,

donded es la dimensión del vector cuantizado antes de la proyección final. Los *codebooks* se inicializan aleatoriamente y ajustados en la etapa de entrenamiento . Para cuantizar un fotograma, una capa lineal proyecta *z* a un conjunto de logits $l \in \mathbb{R}^{G \times V}$, un mecanismo de *Gumbel-softmax* selecciona un índice por grupo, se extraen las filas correspondientes de cada *codebook* y se concatenan en un vector de dimensión d. Finalmente una transformación lineal produce el vector cuantizado q. Esta estructura de *product quantization* permite representar hasta V²=102.400 combinaciones distintas almacenando únicamente $G \times V = 640$ vectores, una ganancia de eficiencia frente a un *codebook* único que requeriría almacenar esas 102.400 combinaciones de forma independiente.

El paso central de este proceso, representado en la Fig. 1 por el bloque del cuantizador, es la elección de una fila en cada *codebook*. Esta elección equivale a tomar el índice de mayor *logit* (una operación $\arg\max$), que no es diferenciable, el gradiente de la pérdida no podría atravesarla y, en consecuencia, ni la capa de *logits* ni los *codebooks* podrían ajustarse. wav2vec 2.0 resuelve este problema mediante *Gumbel-softmax*, que reemplaza la selección rígida por una versión probabilística y suavizada. . Para ello, a cada *logit* se le suma un ruido $\mathrm{n}_v = -\log(-\log u)$, con $\mathrm{u} \sim \mathcal{U}(0,1)$, y se calcula la probabilidad de elegir la entrada v del grupo g como

$$\mathrm{p}_{g,v} = \frac{exp(l_{g,v}+n_v)/\tau}{\sum_{k=1}^{V} exp(l_{g,k}+n_k/\tau)} \quad (1)$$

donde τ es una temperatura que controla qué tan concentrada es la distribución. El ruido introduce exploración, de modo que durante el entrenamiento el modelo no elige siempre la misma entrada y todas las filas del *codebook* tienen oportunidad de actualizarse. La temperatura se reduce progresivamente de 2 a 0,5, de forma que la distribución, inicialmente suave, se vuelve casi determinista al final del preentrenamiento [8]. Finalmente, se aplica un *straight-through estimator*, en el *forward* se usa el índice $\mathrm{i}_g = \arg\max_v p_{g,v}$, de modo que q_t se construye con filas discretas como muestra la figura, mientras que en el *backward* el gradiente se calcula a través de las probabilidades continuas $\mathrm{p}_{g,v}$, lo que permite entrenar los *codebooks* junto con el resto del modelo.

Luego de la etapa de cuantización el modelo se entrena mediante dos pérdidas combinadas. La pérdida contrastiva $\mathcal{L}_m$ evalúa, en cada posición enmascarada, si c_t permite distinguir su versión cuantizada verdadera q_t frente a un conjunto de distractores Q_t (otros q muestreados de otras posiciones enmascaradas del mismo audio), mediante similitud coseno escalada por una temperatura κ :

$$\mathcal{L}_m = -\log \frac{\exp\left(\mathrm{sim}(c_t, q_t)/\kappa\right)}{\sum_{\tilde{q}\in Q_t} \exp\left(\mathrm{sim}(c_t, \tilde{q})/\kappa\right)} \quad (2)$$

donde sim es la similitud coseno, Q_t reúne a q_t y a los $\mathrm{K} = 100$ distractores, y $\kappa = 0{,}1$ [8].

Una pérdida de diversidad $\mathcal{L}_d$ evita que el cuantizador colapse en el uso repetido de solo unas pocas filas de cada *codebook*, maximizando la entropía del uso promedio a lo largo del *batch*:

$$\mathcal{L}_d = \frac{1}{GV}\sum_{g=1}^{G} -H(\bar{p}_g) = \frac{1}{GV}\sum_{g=1}^{G}\sum_{v=1}^{V} \bar{p}_{g,v}\log \bar{p}_{g,v} \quad (3)$$

Donde $\bar{p}_g$ es la probabilidad de selección de cada fila, promediada sobre los fotogramas del *batch*. La función objetivo total combina ambas pérdidas $\mathcal{L} = \mathcal{L}_m + \alpha\mathcal{L}_d$, donde α regula el balance entre utilidad y diversidad de las unidades seleccionadas.

Terminado el preentrenamiento, el cuantizador, los dos *codebooks* y ambas pérdidas se descartan. Lo único que se conserva y se distribuye en el modelo público es el *feature encoder* y el *encoder Transformer*, más una etapa de clasificación añadida que dependera del fenomeno en el que estemos interesados analizar. La Fig. 2 muestra esta configuración: el *feature encoder* (a) transforma el audio crudo en la secuencia de vectores latentes Z; el *encoder* Transformer (b) la proyecta a 768 dimensiones y produce las representaciones contextuales C, que se resumen en un único vector mediante la media temporal, y la etapa de clasificación (c) asigna finalmente la clase correspondiente.

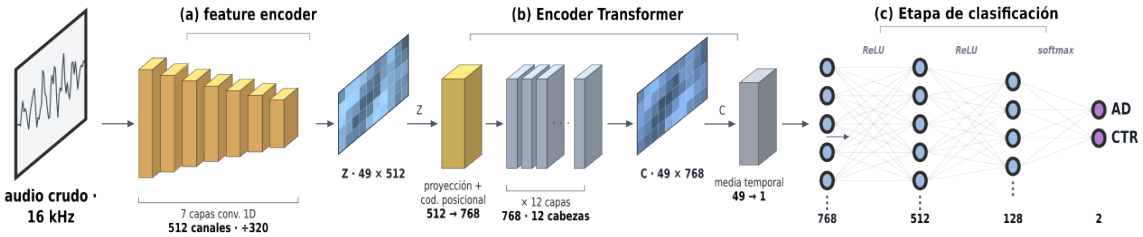


Fig. 2: Arquitectura de wav2vec 2.0 para clasificación: (a) feature encoder, (b) encoder Transformer y (c) etapa de clasificación.

2) feature encoder:

El feature encoder es la primera fase de la arquitectura de wav2vec 2.0 y opera directamente sobre el audio crudo, sin ningún paso intermedio de extracción de características tipo espectrograma. Es la misma etapa que produce la secuencia de vectores latentes z descrita en la sección anterior: siete bloques convolucionales 1D, cada uno con 512 canales, con anchos de kernel (10, 3, 3, 3, 3, 2, 2) y strides (5, 2, 2, 2, 2, 2, 2) [7].

El producto de estos siete strides (5×2×2×2×2×2×2 = 320) determina el factor de *downsampling* temporal total de la capa: por cada 320 muestras de audio crudo a la entrada, el *feature encoder* produce un único vector z a la salida. Para un audio muestreado a 16 kHz, esto se traduce en una tasa de salida aproximada de 49 Hz, es decir, un vector cada ~20 ms, de modo que cada posición de z resume una ventana corta y solapada de la señal cruda, sin ningún tipo de contexto, el cual se incorpora más adelante, en el *encoder Transformer*.

3) encoder transformer:

El *encoder* Transformer de wav2vec 2.0 está formado por una pila de capas idénticas, cuyo número depende de la versión del modelo, la versión base tiene 12 capas con una dimensión oculta de 768 y 12 cabezas de atención, mientras que la versión grande tiene 24 capas con una dimensión de 1024 [8]. Antes de ingresar a la primera capa la secuencia z producida por el *feature encoder*, se proyecta de 512 a 768 dimensiones y se le suma una codificación posicional relativa, implementada como una capa convolucional (kernel de tamaño 128, 16 grupos) en lugar de los embeddings posicionales sinusoidales fijos habituales en el Transformer original. Esto es

necesario porque la atención, a diferencia de una convolución o una recurrencia, no distingue por sí sola el orden de los elementos de la secuencia: sin esta señal, el modelo trataría a z como un conjunto no ordenado de vectores.

Cada una de las 12 capas del encoder repite el mismo bloque (Fig. 3), compuesto por los siguientes elementos:

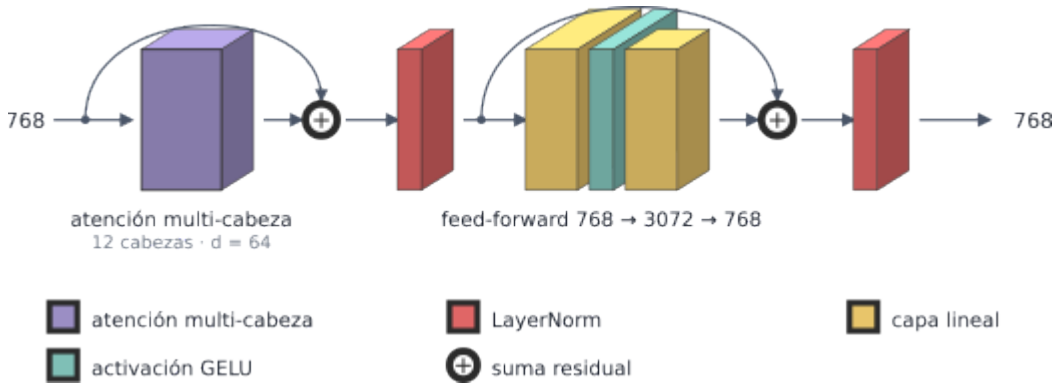


Fig. 3. Estructura de una capa del encoder Transformer de wav2vec 2.0.

a) Atención multi-header

Es el mecanismo que introduce contexto: cada posición, actuando como *query*, calcula su similitud contra las *keys* de todas las demás posiciones de la secuencia mediante el producto punto escalado; esos puntajes se normalizan con una función *softmax* y la salida de esa posición se reconstruye como una suma ponderada de los *values* de todas las posiciones, dando más peso a aquellas con las que tiene mayor similitud. Que sea "multi-cabeza" (12 cabezas de d=64 cada una, sumando 768) significa que esta operación se repite en paralelo 12 veces, cada una sobre una proyección distinta del mismo vector; esto permite que cada cabeza se especialice en capturar un tipo distinto de relación entre diferentes posiciones.

b) Suma residual.

La salida de las cabezas de atención no reemplaza la entrada original del bloque, con el fin de evitar que la información original del fotograma se pierda al pasar por la atención y facilitar la propagación del gradiente durante el entrenamiento, el contenido original que entra al *encoder s*e suma con la salida contextualizada. Sin esta suma, el gradiente tendería a degradarse a medida que la señal atraviesa las 12 capas.

c) LayerNorm.

Normaliza el vector resultante (media 0, varianza 1 a lo largo de la dimensión de características) antes de continuar. Esto estabiliza la magnitud de las activaciones capa tras capa, evitando que crezcan o se degraden de forma descontrolada conforme la señal avanza por las 12 capas.

d) Feed-forward.

Es una red de dos capas lineales aplicada de forma independiente a cada posición de la secuencia: primero expande la dimensión de 768 a 3072, aplica una función no lineal GELU, y luego proyecta de vuelta a 768. Mientras que la atención decide de qué otras posiciones tomar información, la feed-forward transforma el contenido de cada posición de forma no lineal, añadiendo capacidad de representación que la atención por sí sola no provee.

Luego del *feed-forward* se aplica nuevamente una suma residual que pasa por una capa de normalización cerrando el bloque que compone al *encoder transformer.* Apilando 12 de estos bloques se obtiene, para cada fotograma de la secuencia, una representación contextualizada C (49×768) que a diferencia de z ,que solo resume una ventana corta de audio crudo, ya incorpora información de toda la secuencia gracias a las 12 rondas sucesivas de atención.

3) Etapa de clasificación:

wav2vec2.0 se evalúa fundamentalmente como modelo base para reconocimiento automático de habla (ASR) [8]. Para esto, el *header* que se coloca sobre el Transformer preentrenado es una única capa lineal, inicializada aleatoriamente, que proyecta cada vector de la secuencia contextualizada C a un conjunto de logits sobre el vocabulario de la tarea. Esta etapa conserva la longitud temporal de C (produce una predicción por cada fotograma, no una única predicción por audio) porque el problema que resuelve es de secuencia a secuencia sin un alineamiento conocido entre fotogramas de entrada y símbolos de salida; por eso se entrena con la pérdida CTC (Connectionist Temporal Classification), que evita el alineamiento sumando todas las segmentaciones posibles compatibles con la secuencia a encontrar.

Adaptar wav2vec 2.0 a una tarea de clasificación implica reutilizar el modelo preentrenado, descartando el cuantizador (empleado solo en preentrenamiento) y reemplazando la etapa de CTC por una nueva dado que C tiene longitud variable y una tarea de clasificación requiere una única predicción por audio. Debido a esto es necesario colapsar C a un vector de dimensión fija antes de clasificar. Esto se resuelve con una operación de *pooling* temporal seguida de una o más capas que proyectan ese vector fijo a K clases. Por lo tanto, la pérdida cambia de la formulación contrastiva de preentrenamiento a una entropía cruzada estándar

$$\mathcal{L}_{CE} = -\sum_{c} y_c \log(\hat{y}_c) \tag{4}$$

Donde y_c es la etiqueta verdadera y $\hat{y}_c$ la probabilidad predicha para la clase c.

4) Interpretabilidad por capas de representación auto-supervisadas:

Si bien el desempeño de estos modelos está bien documentado en diferentes tareas, esto no implica que se conozca qué tipo de información codifica cada capa individualmente, ni como esa información varía a lo largo de la profundidad de la red.

Existen dos aproximaciones generales para responder esta pregunta. La primera consiste en entrenar un clasificador auxiliar ("sonda" o *probe*) sobre la representación de cada capa, y evaluar su desempeño en una tarea específica (p. ej., clasificación de fonemas) [12]. El problema de este enfoque es que introduce una incertidumbre adicional: si el clasificador obtiene un desempeño bajo, no es posible determinar si esto se debe a que la capa efectivamente no codifica la información buscada, o a decisiones de diseño del propio clasificador (arquitectura, hiperparámetros, cantidad de datos de entrenamiento). Pasad et al. (2021)[12] optan en su lugar por un segundo tipo de análisis basado en medidas estadísticas que comparan directamente la representación de una capa con una variable externa de interés, sin necesidad de entrenar ningún modelo adicional. En particular, emplean dos medidas complementarias. La primera es una variante del análisis de correlación canónica, *projection-weighted CCA* (PWCCA), que cuantifica la relación lineal entre las representaciones de una capa y una propiedad de referencia como la identidad fonética. La segunda es la información mutua, que captura dependencias de cualquier tipo entre la representación de una capa y etiquetas discretas de fonemas o palabras. Con ambas medidas aplicadas capa por capa, los autores obtienen un perfil de qué tipo de información codifica cada nivel del Transformer.

En la versión base de wav2vec 2.0, los autores identifican una progresión jerárquica de la información a lo largo de las 12 capas del Transformer. En las capas intermedias, entre la 5 y la 7, se concentra el contenido fonético, allí alcanzan su máximo tanto la información mutua con la identidad de los fonemas como la similitud con representaciones de palabras diseñadas para capturar su forma acústico-fonética. Alrededor de las capas 7 y 8, predomina el contenido semántico, ya que la similitud con representaciones del significado de las palabras alcanza su máximo en ese rango. Finalmente, en las capas 9 a 11 la información fonética y léxica disminuye de forma marcada y las representaciones vuelven a parecerse a las de la entrada. Los autores interpretan este comportamiento como el de un *autoencoder*, en el que las primeras capas se alejan progresivamente de las características acústicas locales para construir abstracciones lingüísticas, y las últimas regresan hacia ellas [12].

B. Parameter-Efficient Fine-Tuning(PEFT)

La forma habitual de adaptar un modelo preentrenado como *wav2vec 2.0* a una tarea específica es el *fine-tuning* completo: se parte de los pesos aprendidos en el preentrenamiento y se continúa entrenando la totalidad de ellos con los datos etiquetados de la nueva tarea. Si w_0 son los pesos preentrenados, el fine-tuning completo aprende para cada tarea una actualización Δw del mismo tamaño que w_0, en consecuencia, cada tarea requiere almacenar una copia completa del modelo [17]. Adicionalmente, cuando tenemos un conjunto de datos pequeño para la tarea objetivo, ajustar la red en su totalidad puede resultar subóptimo [16], y reentrenar los pesos compartidos expone al modelo a olvidar las representaciones aprendidas previamente, fenómeno conocido como olvido catastrófico [16].

El ajuste fino eficiente en parámetros (PEFT, por sus siglas en inglés) aborda estas limitaciones manteniendo congelados los pesos preentrenados y entrenando únicamente un conjunto pequeño de parámetros nuevos. Houlsby et al [16], lo formalizan de la siguiente manera: dada una red $\varphi_w(x)$ con parametros preentrenados w, se define una nueva función $\psi_{w,v}(x)$ donde w se copía del preentrenamiento y permanece fijo mientras v es un conjunto de parámetros nuevos con $|v| \ll |w|$. Los parámetros iniciales v_0 se eligen de modo que la nueva función reproduzca la original, de forma tal, que durante el entrenamiento solo se ajusta v.

$$\psi_{w,v_0}(x) \approx \varphi_w(x) \quad (5)$$

Esta condición de inicialización garantiza que el entrenamiento arranque exactamente desde el comportamiento del modelo preentrenado y se aleje de él de forma gradual desencadenando en dos atributos principales: el modelo resultante es compacto porque cada tarea añade solo unos pocos parámetros y adicionalmente es extensible debido a que se pueden incorporar tareas nuevas sin alterar los pesos compartidos con las tareas o dimensiones previamente entrenadas.

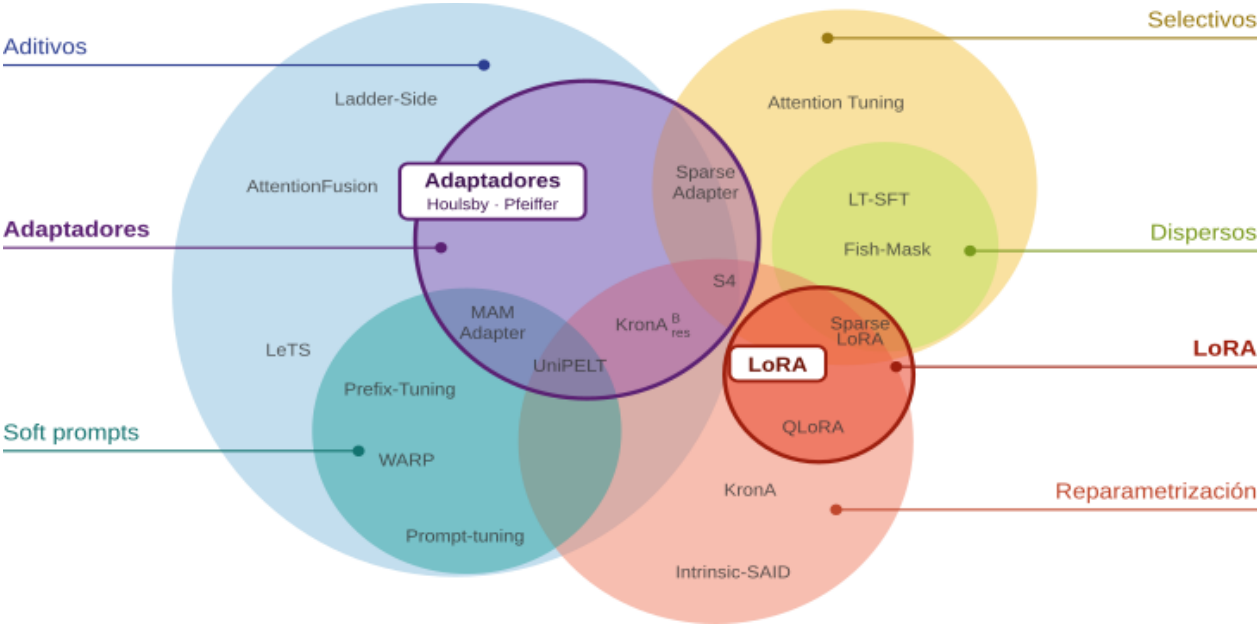


Fig. 4: Taxonomía de métodos PEFT, con los adapters y LoRA resaltados. Adaptada de Lialin et al. [20].

Las técnicas PEFT pueden clasificarse en tres clases principales según sea su enfoque [20]: los métodos aditivos, que aumentan el modelo preentrenado con parámetros o capas nuevas y entrenan únicamente esos elementos; los métodos selectivos, que ajustan un subconjunto de los parámetros ya existentes del modelo, como las últimas capas de la red o únicamente sus sesgos; y los métodos de reparametrización, que aprovechan representaciones de bajo rango para expresar la actualización de los pesos con un número reducido de parámetros entrenables. A estas se suman los métodos híbridos, que combinan ideas de varias clases y aparecen en las intersecciones de la Fig. 4, que resume esta taxonomía . Dentro de los métodos aditivos, a su vez, se distinguen dos grandes subgrupos: los métodos tipo adapter, que insertan pequeñas redes totalmente conectadas después de las sub-capas del Transformer, y los soft prompts, que ajustan mediante gradiente una parte de los *embeddings* de entrada del modelo. Estas técnicas buscan reducir el consumo de memoria, mejorar la eficiencia de almacenamiento o aportar modularidad, y difieren entre sí en la cantidad de parámetros entrenables y el costo de entrenamiento e inferencia [20].

1) Adaptadores (Adapters):

Un adapter es un módulo pequeño de parámetros nuevos que se inserta dentro de las capas de una red preentrenada cuyos pesos originales permanecen congelados; durante el entrenamiento solo se ajustan los parámetros del adapter [16]. A este modulo se le atribuyen dos características principales: un número reducido de parámetros en comparación con las capas de la red original, lo que hace que el tamaño total del modelo crezca lentamente a medida que se añaden tareas, y una inicialización cercana a la identidad, necesaria para que el entrenamiento sea estable.

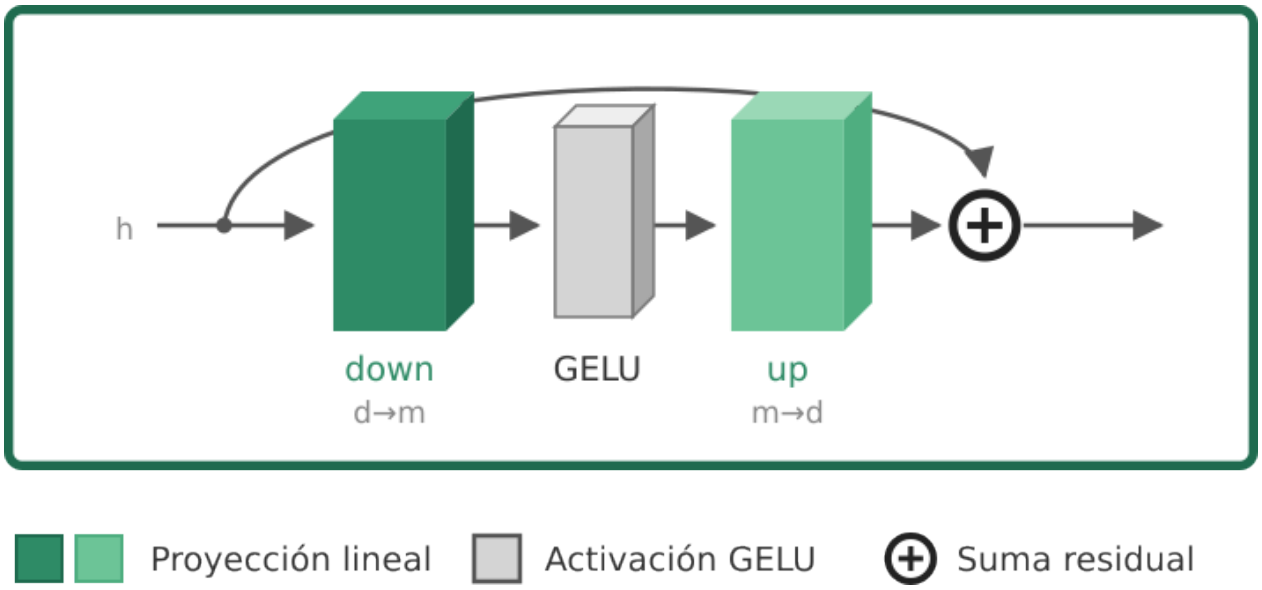


Fig. 5: Estructura interna de un adapter bottleneck

Para limitar el número de parámetros, el adapter adopta una arquitectura de cuello de botella (Fig. 5). Dada una representación de entrada $\mathrm{h} \in \mathbb{R}^d$, una primera capa lineal (FF Down) la proyecta a una dimensión menor m, Sobre esta se aplica una función no lineal f y una segunda capa lineal (FF Up) la proyecta de vuelta a las d dimensiones originales. El módulo incluye además una conexión residual interna que suma la entrada a esa transformación:

$$\mathrm{Adapter}(h) = h + W_{up}\, f(W_{down}\, h + b_{down}) + b_{up} \quad (6)$$

Donde $\mathrm{W}_{down} \in \mathbb{R}^{m \times d}$ y $b_{down} \in \mathbb{R}^{m}$ son los parámetros de FF Down, y $\mathrm{W}_{up} \in \mathbb{R}^{d \times m}$ y $b_{up} \in \mathbb{R}^{d}$ los de FF Up. El número total de parámetros que añade cada adapter, incluyendo los sesgos es de $2md + d + m$.

La suma residual interna es la que permite la inicialización cercana a la identidad: si los parámetros de ambas proyecciones se inicializan con valores próximos a cero, el término $\mathrm{W}_{up}\, f(W_{down}\, h + b_{down}) + b_{up}$ es aproximadamente nulo y el módulo se reduce a $\mathrm{Adapter(h)} \approx h$ [16]. De este modo, la red original no se ve afectada al inicio del entrenamiento, y es durante este que los adapters se activan para modificar las activaciones a lo largo de la red.

Al fijar $\mathrm{m} \ll d$, este número se mantiene pequeño; en la práctica, se utiliza entre el 0.5 y el 8% de los parámetros del modelo original [16] . Esta matriz se expresa mediante el factor de reducción r, que indica cuántas veces se reduce la dimensión oculta en el cuello de botella [16] $r = \frac{d}{m}$. Por ejemplo, para una dimensión oculta $\mathrm{d} = 768$, un factor $\mathrm{r} = 64$ produce un cuello de botella de $\mathrm{m} = 12$ dimensiones [21].

2). Low-Rank Adaptation (LoRA):

A diferencia de los adapters, que introducen módulos nuevos en la red, LoRA pertenece a los métodos de reparametrización, es decir, no añade capas extra al modelo, en su lugar, restringe la forma que puede tomar la actualización de los pesos ya existentes [10]. Este metodo se fundamenta en el comportamiento del rango $rank(W)$ de las matrices que se utilizaran para la actualización que se define como la dimensión del espacio generado por sus columnas, en otras palabras, el rango es el número máximo de columnas linealmente independientes que tiene una matriz.

La factorización de rango establece que toda matriz $\mathrm{M} \in \mathbb{R}^{d \times k}$ con $\mathrm{rank}(M) = r$ puede escribirse como el producto de dos matrices de dimensión intermedia r:$M = BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}$ como se muestra en Fig. 6. donde las r columnas de B forman una base del espacio columna de M, y cada columna de A contiene los coeficientes que

expresan la columna correspondiente de M en esa base [22]. El número de valores necesarios para describir M pasa así de $d \times k$ a $\mathrm{r(d + k)}$, una reducción considerable cuando $\mathrm{r} \ll \min(d, k)$.

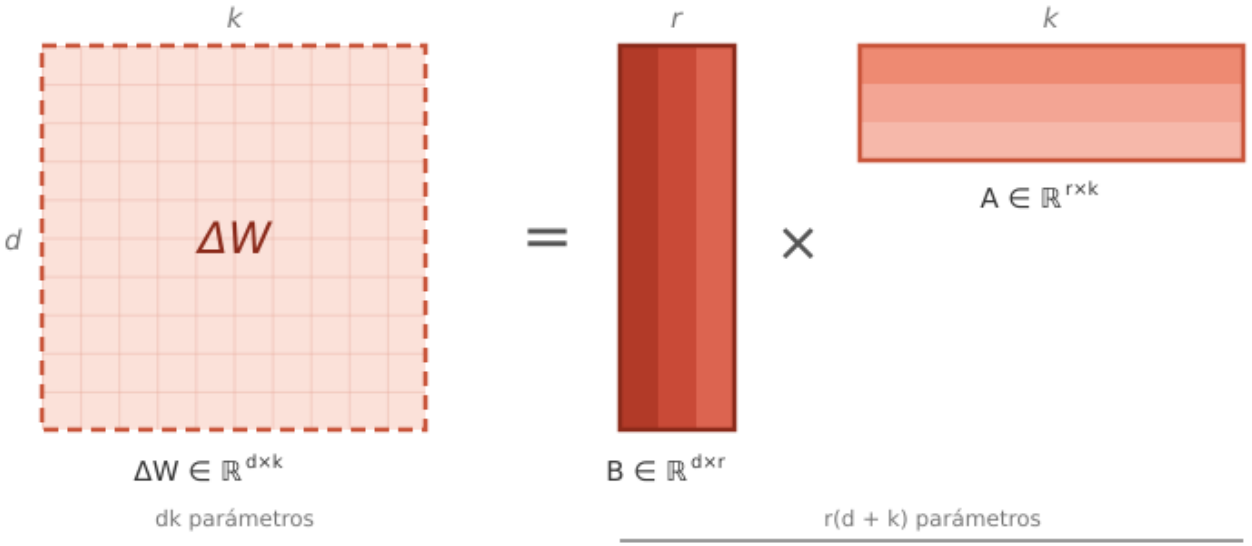


Fig. 6: Factorización de rango de la actualización de pesos ΔW en las matrices de bajo rango B y A.

Recíprocamente, cualquier producto de esta forma tiene rango a lo sumo r, ya que $\mathrm{rank}(BA) \leq \min(\mathrm{rank}(B), \mathrm{rank}(A)) \leq r$. De esta manera aunque la matriz sea formalmente de rango completo, su contenido relevante vive en un subespacio de dimensión r. Hu et al. [10] se inspiran en trabajos previos que muestran que los modelos preentrenados pueden adaptarse eficientemente aun cuando el entrenamiento se restringe a un subespacio de parámetros de dimensión reducida [10]. A partir de ello formulan la hipótesis central de LoRA: la actualización de los pesos durante la adaptación tiene un "rango intrínseco" bajo [10]. Es decir, para cada matriz preentrenada W_0, la actualización ΔW que la adapta a la tarea, aunque sea de la misma dimensión $\mathrm{d} \times k$, concentra su contenido relevante en unas pocas direcciones, de modo que se cumple:

$$\mathrm{rank}(\Delta W) = r \ll \min(d, k) \quad (7)$$

Si esta hipótesis se cumple, no es necesario aprender ΔW entrada por entrada. Que esta matriz tenga rango r significa que todas sus columnas son combinaciones lineales de solo r vectores base. Por la factorización de rango, basta entonces con aprender dos matrices pequeñas: $\mathrm{B} \in \mathbb{R}^{d \times r}$, cuyas columnas son esos r vectores base, y $\mathrm{A} \in \mathbb{R}^{r \times k}$, que contiene, para cada columna de ΔW, los r coeficientes con los que se combinan dichos vectores. Su producto reconstruye la actualización completa:

$$\Delta W = BA \quad (8)$$

El número de parámetros a aprender pasa así de $\mathrm{d} \times k$ a $\mathrm{r}(\mathrm{d} + \mathrm{k})$, las entradas de B y A. Por ejemplo, para $\mathrm{d} = \mathrm{k} = 1024$ y $\mathrm{r} = 8$, esto supone pasar de 1.048.576 a 16.384 parámetros, aproximadamente el 1.6% del total.

Por lo tanto, Dada una matriz preentrenada $\mathrm{W}_0 \in \mathbb{R}^{d \times k}$, LoRA la mantiene congelada y suma a su salida la contribución de la actualización de bajo rango $\Delta W = BA$, escalada por un factor α / r [10]:

$$h = W_0 x + \frac{\alpha}{r} BAx \quad (9)$$

donde B y A son los únicos parámetros entrenables. El rango r determina la dimensión interna de la actualización y controla directamente el compromiso entre la capacidad de adaptación y el número de parámetros. Por su parte, α es un factor de escala constante que regula la magnitud con la que la actualización BA se suma a la salida preentrenada. A se inicializa con valores gaussianos aleatorios y B con ceros, de modo que $\Delta W = 0$ al inicio y la capa reproduce exactamente su comportamiento preentrenado.

$A_q \in \mathbb{R}^{r \times d}$ipio, LoRA puede aplicarse a cualquier matriz densa de la red [10]. En una capa Transformer, las candidatas son las cuatro proyecciones del módulo de atención (W_q, W_k, W_v, W_o) y las dos del módulo feed-forward. Adaptar una proyección de atención, por ejemplo W_q, consiste en sustituir $\mathrm{W}_q x$ por $W_q x + \frac{\alpha}{r} B_q A_q x$, con $B_q \in \mathbb{R}^{d \times r}$ y $A_q \in \mathbb{R}^{r \times d}$; el resto del mecanismo de atención no cambia, añadiendo al modelo $2\,r\,d$ parámetros.

IV. BASE DE DATOS

Se utilizó la base de datos del reto ADReSS, presentado en Interspeech 2020. Sus grabaciones provienen del Pitt Corpus de DementiaBank [23]. Los participantes describen la lámina *Cookie Theft* del Boston Diagnostic Aphasia Examination (BDAE), una tarea ampliamente usada en la evaluación del lenguaje en poblaciones con deterioro cognitivo.

A diferencia del Pitt Corpus completo, que presenta un desbalance entre grupos, ADReSS fue construido específicamente para estar balanceado por edad y género entre los sujetos con AD y los controles. Este balance reduce la posibilidad de que el clasificador aprenda variables demográficas en lugar de características asociadas a la enfermedad, y convierte a ADReSS en un punto de referencia comparable entre estudios. La base de datos contiene 156 sujetos en total, 78 con diagnóstico de Alzheimer (AD) y 78 controles sanos (CTR), divididos por los organizadores en una partición de entrenamiento de 108 sujetos y una de prueba de 48, ambas balanceadas por clase (54 AD y 54 CTR en entrenamiento; 24 AD y 24 CTR en prueba).

Al contener únicamente segmentos con actividad de voz, esta versión elimina los silencios prolongados y las intervenciones sin habla, y garantiza un nivel de amplitud homogéneo entre sujetos, lo que reduce la influencia de las condiciones de grabación sobre la clasificación.

Además, para la optimización de hiperparámetros, la partición oficial de entrenamiento se dividió en un subconjunto de entrenamiento y uno de validación en proporción aproximada de 80/20. Para ello se empleó un esquema de particionado estratificado y agrupado por sujeto en cinco particiones, con una semilla fija, del cual se tomó únicamente la primera partición como conjunto de validación, es decir, se obtuvo una única división, sin validación cruzada. La agrupación garantiza que todos los fragmentos de un mismo sujeto queden en un solo subconjunto, evitando la fuga de información entre entrenamiento y validación, mientras que la estratificación se aplicó sobre la proporción de fragmentos de cada clase, por lo que el balance a nivel de sujeto es aproximado. El resultado fue un conjunto de entrenamiento de 87 sujetos (45 AD y 42 CTR) y uno de validación de 21 sujetos (9 AD y 12 CTR). Esta división se mantuvo idéntica en todos los escenarios experimentales, y la partición oficial de prueba se reservó exclusivamente para la evaluación final de los modelos seleccionados.

V. METODOLOGÍA

Este capítulo describe el procedimiento experimental seguido para evaluar el efecto de distintas estrategias de ajuste de un modelo de habla autosupervisado preentrenado sobre la clasificación binaria entre sujetos con enfermedad de Alzheimer y controles sanos a partir de habla espontánea.

La Figura 6 presenta el flujo general de la metodología propuesta. Las grabaciones del corpus ADReSS 2020 se remuestrean a 16 kHz y se dividen en ventanas de duración fija. Cada ventana se procesa con wav2vec 2.0, sobre el cual se aplican tres estrategias de ajuste evaluadas: descongelamiento parcial, adaptadores *bottleneck* y LoRA. Las representaciones de la última capa se agregan mediante la media temporal, lo que produce un vector de dimensión 768 por ventana. Este vector se clasifica a partir de una red completamente conectada . Finalmente, las predicciones de las ventanas de cada sujeto se combinan mediante votación mayoritaria para asignar la etiqueta final por sujeto: AD o CTR.

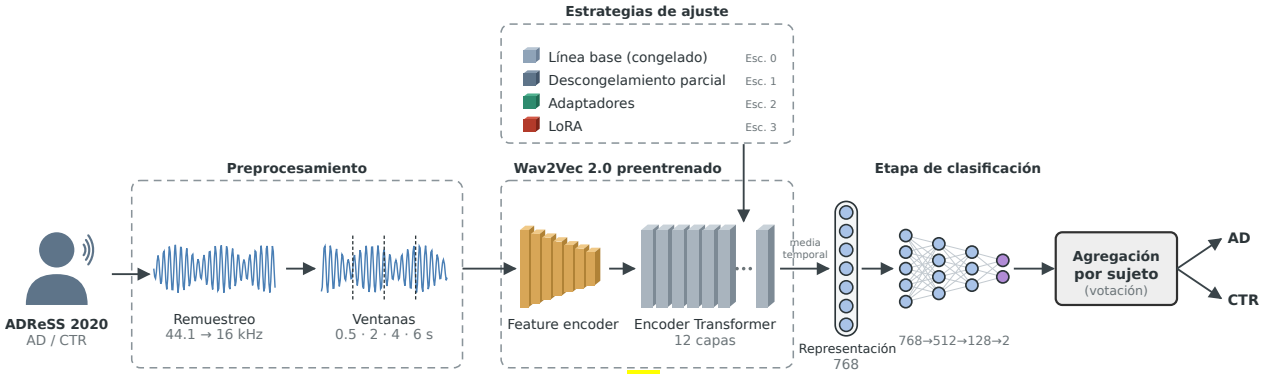


Fig. 7: Metodologia implementada

A. Preprocesamiento:

1) Remuestreo

Las grabaciones de ADReSS se encuentran muestreadas a 44.1 kHz, mientras que wav2vec 2.0 fue preentrenado con audio a 16 kHz. Por lo tanto, cada segmento se convirtió a un único canal, promediando sus canales cuando la señal era estéreo, y se remuestreó de 44.1 kHz a 16 kHz.

2) Enventanado

De manera inicial, se evaluó como condición de referencia el uso directo de los segmentos normalizados de ADReSS, sin concatenación ni enventanado, que corresponde al procedimiento habitual al trabajar con esta base de datos. Dichos segmentos entregados por los organizadores corresponden a fragmentos de habla activa de duración variable, con un máximo de 10 s.

Adicionalmente, se construyó para cada sujeto una única señal continua mediante la concatenación de todos sus segmentos en orden cronológico. Luego, la señal de cada sujeto se dividió en ventanas de duración fija con un traslape del $25\,\%$. La porción final de la señal que no alcanza a completar una ventana se descartó en lugar de completarse con ceros. Esta decisión se sustentó en un análisis de sensibilidad en el que se evaluó la inclusión de la ventana final incompleta cuando superaba un umbral mínimo de duración, rellenando el resto con ceros, dicha alternativa incrementaba el número de ventanas en aproximadamente un $2\,\%$, a cambio de introducir segmentos de silencio artificial que contaminan la representación. Como consecuencia adicional, todas las ventanas de un mismo experimento tienen exactamente la misma longitud, por lo que no se requiere relleno al conformar los *batchs* de entrenamiento.

Respecto a la duración, con el objetivo de evaluar el efecto del contexto temporal disponible para el modelo, se generaron conjuntos de ventanas con duraciones entre 0.5 s y 7.5 s. El límite inferior corresponde a una escala cercana a la duración de unidades fonéticas y silábicas, mientras que el límite superior está acotado por la duración mínima de la base de datos, la cual corresponde a 7.89 s. Esto garantiza que se tienen la misma cantidad de participantes sin

importar el tamaño de la ventana. Dado el costo computacional de la metodología planteada, no todas las duraciones generadas se evaluaron en la totalidad de los experimentos. En su lugar, se realizó un barrido inicial con el escenario congelado, es decir, usando wav2vec 2.0 solo como extractor de características sobre todas las ~~duraciones de~~ ventana. El *baseline* se utilizó para esta selección por tratarse del escenario de menor costo y porque, al mantener el modelo congelado, refleja la calidad de las representaciones obtenidas con cada duración sin depender de la estrategia de ajuste . Posteriormente, el resultado con mejor desempeño, es al cual se le aplican las diferentes estrategias de adaptación planteadas en la metodología.

C. Elección del modelo de wav2vec 2.0

Se evaluaron seis variantes preentrenadas de wav2vec 2.0 que difieren en su capacidad, en el volumen y la naturaleza de los datos de preentrenamiento, en la cobertura de idiomas y en si fueron posteriormente ajustadas para reconocimiento automático del habla. La Tabla I resume sus características principales.

El conjunto de candidatos permite contrastar tres ejes de variación: el tamaño del modelo (Base frente a Large), el volumen y la diversidad de los datos de preentrenamiento (desde 960 h de lectura en inglés hasta 436 000 h en 128 idiomas) y el efecto de un ajuste posterior para ASR sobre las representaciones.

VoxPopuli base y XLSR-300M obtuvieron la mayor exactitud a nivel de sujeto (68.8 %), por encima del resto de variantes (entre 60.4 % y 66.7 %). Se eligió VoxPopuli porque alcanza el mismo desempeño con cerca de 95 millones de parámetros, aproximadamente la tercera parte de los 300 millones de XLSR-300M, lo que lo hace más adecuado para un escenario con pocos datos.

TABLA I: RESULTADO DE EXPERIMENTOS PARA VENTANAS DE 7.5 SEGUNDOS

Modelo	**Parámetros**	**Capas Transformer**	**Dimensión oculta**	**Datos de preentrenamiento**	**Ajuste para ASR**	**Idiomas**
wav2vec2 BASE	~95 M	12	768	960 h (LibriSpeech)	No	Inglés
wav2vec2 ASR BASE 960H	~95 M	12	768	960 h (LibriSpeech)	960 h (LibriSpeech)	Inglés
wav2vec2 LARGE LV60K	~317 M	24	1024	60 000 h (Libri-Light)	No	Inglés
wav2vec2 ASR LARGE LV60K 960H	~317 M	24	1024	60 000 h (Libri-Light)	960 h (LibriSpeech)	Inglés
wav2vec2 XLSR 300M	~300 M	24	1024	436 000 h	No	128
VOXPOPULI ASR BASE 10K EN	~95 M	12	768	10 000 h (VoxPopuli)	543 h (VoxPopuli, inglés)	23 en preentrenamiento inglés en el ajuste

Dado que todas las estrategias de ajuste evaluadas en este trabajo se aplican sobre un único modelo preentrenado, la selección de dicho modelo constituye una etapa previa al diseño experimental. Para ello, los modelos candidatos se compararon completamente congelados y entrenando únicamente la etapa de clasificación, de modo que la dicha comparación refleje la calidad de las representaciones de cada modelo para la tarea, sin que el resultado dependa de la estrategia de ajuste.

El modelo seleccionado fue VOXPOPULI ASR BASE 10K EN [24] . Este modelo emplea la arquitectura Base de wav2vec 2.0 que fue preentrenado de forma autosupervisada con 10 000 h de habla no etiquetada del corpus VoxPopuli, que reúne grabaciones de las sesiones del Parlamento Europeo en 23 idiomas, y posteriormente ajustado para ASR con 543 h de habla transcrita en inglés del mismo corpus.

D. Estrategias de ajuste

Todos los escenarios comparten la etapa de clasificación, la función de pérdida de entropía cruzada y el mismo espacio de búsqueda para los hiperparámetros: tres tasas de aprendizaje, $\{10^{-3}, 10^{-4}, 10^{-5}\}$, y tres coeficientes de regularización (*weight decay*), $\{10^{-2}, 10^{-3}, 10^{-4}\}$. A este espacio se suman, en los mecanismos de adaptadores y LoRA, los hiperparámetros propios de cada uno. La línea base, que entrena únicamente la etapa de clasificación sobre características congeladas, se optimizó con Adam, mientras que los escenarios que modifican el modelo preentrenado (descongelamiento parcial, adaptadores y LoRA) emplearon AdamW, que aplica el *weight decay* de forma desacoplada de la actualización del gradiente. El entrenamiento se extendió hasta un máximo de 200 épocas y se detuvo de forma anticipada cuando la pérdida en validación no mejoró en más de 10^{-4} durante 20 épocas consecutivas, conservando los pesos de la época con menor pérdida. En la evaluación, la etiqueta de cada sujeto se asignó mediante votación mayoritaria sobre las predicciones de sus ventanas. El desempeño final se reporta a nivel de sujeto sobre los 48 sujetos de prueba, considerando AD como clase positiva, mediante la exactitud, la sensibilidad, la especificidad, el F1 y el AUC, este último calculado a partir de la probabilidad promedio de la clase AD por sujeto.

Las estrategias de ajuste se organizaron de acuerdo a dos factores principales. El primero corresponde al mecanismo de ajuste, con tres niveles: descongelamiento parcial de las capas, inserción de módulos adaptadores de tipo *bottleneck* y adaptación de bajo rango (LoRA). El segundo corresponde al tipo de experimento, que define en qué capas del Transformer se aplica el ajuste, también con tres niveles:

- Estándar: el ajuste se distribuye en un conjunto fijo de capas equidistantes, concretamente las capas 3, 6, 9 y 12.
- Progresivo: el ajuste se aplica a las N últimas capas, con N = 1, …, 12.
- Inteligente: el ajuste se restringe a los bloques funcionales identificados por Pasad et al. (2021)[11] que agrupan las capas del Transformer según el tipo de información que

codifican predominantemente: el bloque A (capas 6–7) asociado al contenido fonético; el bloque B (capas 8–9) asociado al contenido semántico, y el bloque C (capas 10–12) correspondiente a la fase reconstructiva.

1) Descongelamiento parcial

En este mecanismo se descongelan los pesos originales de las capas seleccionadas bajo las estrategias progresivas e inteligente, dado que la estrategia estándar cobra sentido en las estrategias basadas en adaptadores y LoRA se decidio no evaluarla en el descongelamiento parcial. Dichas capas se actualizan junto con la etapa de clasificación, mientras el resto del modelo permanece congelado. El *feature encoder* se mantuvo congelado en todos los casos. Ademas, dado que los pesos preentrenados son sensibles a actualizaciones de gran magnitud, se emplearon tasas de aprendizaje diferenciadas mediante dos grupos de parámetros. Para las capas descongeladas se fijó una tasa de 1e-5, fuera del espacio de búsqueda. El coeficiente de weight decay se compartió entre ambos grupos, de modo que cada configuración comprendió 9 combinaciones de hiperparámetros.

4) Adapters de tipo bottleneck

En este mecanismo, el modelo permanece completamente congelado y se insertan módulos adaptadores en las capas seleccionadas. Se evaluaron dos esquemas de inserción en los tres tipos de experimento, el de Pfeiffer et al. (2020) (Fig. 7), con un adaptador por capa ubicado después de la red de alimentación directa (feed-forward), y el de Houlsby et al. (2019) (Fig. 8), con dos adaptadores por capa, ubicados después del bloque de atención y después de la red feed-forward. Cada esquema se trató como una variante independiente, con su propia búsqueda de hiperparámetros.

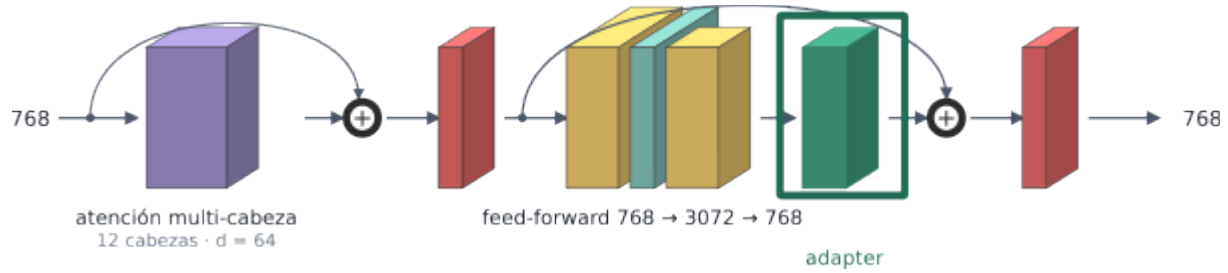


Fig. 8: Adaptador Bottleneck en configuración Pfeiffer.

Cada adaptador proyecta la representación de dimensión d = 768 a una dimensión reducida $m = d/r_f$, donde r_f es el factor de reducción tratado como hiperparámetro con valores $r_f \in \{8, 16, 32\}$, que corresponden a dimensiones de cuello de botella de 96, 48 y 24, respectivamente. Cada adaptador utiliza una función de activación GELU y cuenta con 2dm + m + d parámetros, es decir, 37.656, 74.544 y 148.320 respectivamente.

Los adaptadores se inicializaron cerca de la identidad, es decir, la proyección de subida se inicializó en cero y la de bajada con una distribución normal de desviación estándar 1e-3, esto garantiza que al inicio del entrenamiento el modelo reproduce exactamente las representaciones preentrenadas. Además de los adaptadores y el *header*, se entrenaron los parámetros de normalización de capa (layer normalization) posteriores a cada punto de inserción.

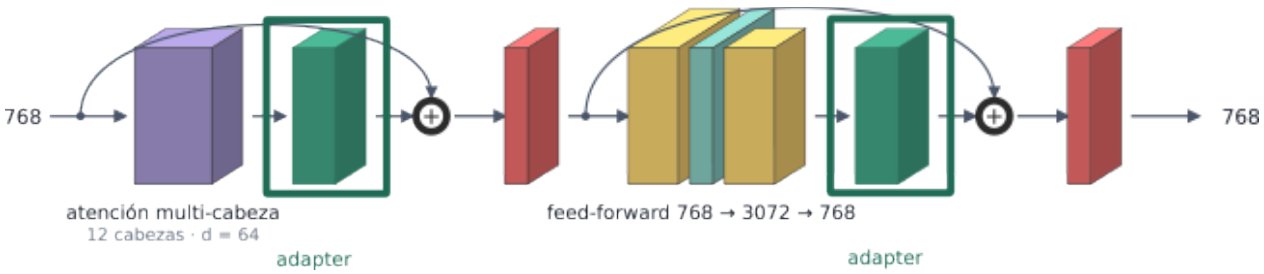


Fig. 9: Adaptador Bottleneck en configuración Houlsby

5) LoRA

En este mecanismo se añaden matrices de bajo rango a proyecciones lineales del mecanismo de atención de las capas seleccionadas. Las proyecciones intervenidas se trataron

como una variable del diseño, con siete variantes. En primer lugar, se evaluaron las proyecciones de consulta (q), clave (k), valor (v) y salida (o) de forma individual, con el fin de aislar la contribución de cada una. En segundo lugar, se evaluaron tres combinaciones de dos proyecciones. La combinación qv corresponde a la configuración propuesta en el trabajo original de LoRA y Las combinaciones qk y vo se definieron a partir de la descomposición funcional del mecanismo de atención propuesta por Elhage et al. (2021) [25]. De este modo, la combinación qk permite adaptar únicamente la forma en que se distribuye la atención, y la combinación vo únicamente el contenido que la atención transmite.

Para cada variante se exploró el rango $r \in 4, 8, 16$, con un factor de escala $\alpha = 2r$. Se aplicó un dropout de 0.1 sobre la entrada de la rama de bajo rango. La matriz B se inicializó en cero y la matriz A con la inicialización uniforme de Kaiming, siguiendo la implementación de referencia de LoRA, de modo que la salida del modelo al inicio del entrenamiento coincide con la del modelo preentrenado. Dado que en wav2vec 2.0 base todas las proyecciones de atención tienen dimensión 768×768, cada matriz adaptada añade $\mathrm{r}(\mathrm{d} + \mathrm{k}) = 2 \cdot 768 \cdot r$ parámetros entrenables, es decir, 6.144, 12.288 y 24.576 para $\mathrm{r} = 4$, 8 y 16, respectivamente. Este número se duplica en las variantes que adaptan dos matrices (qk, qv y vo) y se multiplica por el número de capas en las que se inserta LoRA, de modo que el total de parámetros entrenables de cada configuración se mantiene muy por debajo del ~~de un solo~~ adaptador *bottleneck*.

E. Etapa de clasificación

La representación de cada ventana se clasifica mediante una red totalmente conectada con dos capas ocultas de 512 y 128 neuronas y función de activación ReLU, seguido de una capa de salida de dos neuronas. La misma arquitectura se utilizó en todos los escenarios experimentales, y se inicializó aleatoriamente en cada entrenamiento. En el escenario *baseline*, el *header* se entrena sobre las representaciones extraídas previamente del codificador congelado; en los demás escenarios, se entrena conjuntamente con los parámetros del modelo que se ajustan en cada caso.

V. ANÁLISIS DE RESULTADOS

A. Referencia: Segmentos originales de ADReSS

El análisis de resultados parte de la condición más directa posible: evaluar el *baseline* sobre los segmentos de audio tal como los distribuye el corpus orginal, sin ningún procesamiento adicional. Este resultado permite establecer el punto de comparación contra el cual se mide el efecto del enventanado y de las estrategias de ajuste evaluadas en las secciones siguientes. La Tabla II resume las métricas obtenidas sobre la partición de prueba. En esta primera etapa, se utilizo wav2vec 2.0 completamente congelado, utilizandolo unicamente como extractor de caracteristicas.

TABLA II: DESEMPEÑO SOBRE LOS SEGMENTOS ORIGINALES DE ADReSS

Escenario	Exactitud	Sensibilidad	Especificidad	F1-Score	AUC
Baseline	64.6%	83.3%	45.8%	70.2%	0.80

La exactitud obtenida (64.5 %) es comparable con el desempeño obtenido en el reto original de 62.5% [9]. Sobre los segmentos originales, las representaciones congeladas de wav2vec 2.0 no ofrecen una ventaja sustancial frente a los descriptores acústicos clásicos. Por otro lado, cuando analizamos la relación entre sensibilidad y especificidad, observamos que el aspecto más relevante es el desbalance entre ambas metricas de desempeño. El modelo identifica correctamente 20 de los 24 sujetos con AD, pero solo 11 de los 24 controles, de modo que más de la mitad de los controles son clasificados como pacientes.

B. Enventanado

Una vez establecido el desempeño sobre los segmentos originales, se evaluó cómo cambia el comportamiento del modelo completamente congelado cuando estos se sustituyen por ventanas

de diferentes longitudes. Este barrido permite identificar las duraciones de ventana en las que las representaciones de wav2vec 2.0 contienen información suficiente para discriminar entre pacientes con AD y controles, y con ello definir el tamaño de la entrada sobre las que se evalúan las estrategias de ajuste. La Tabla III presenta las metricas obtenidas para cada duración.

TABLA III: DESEMPEÑO DEL BASELINE POR TAMAÑO DE VENTANA

Tamaño de ventana	**Exactitud**	**Sensibilidad**	**Especificidad**	**F1**	**AUC**
0.5 s	50.0%	0.0%	100.0%	0.0%	0.28
2 s	50.0%	0.0%	100.0%	0.0%	0.45
4 s	50.0%	0.0%	100.0%	0.0%	0.55
6 s	66.7%	58.3%	75.0%	63.6%	0.70
7.5 s	72.9%	66.7%	79.2%	71.1%	0.75

Los resultados muestran dos tamaños de ventana claramente diferenciados. En el caso de ventanas de 6 s de duración la exactitud alcanza 66.7%, ligeramente por encima de la obtenida con los segmentos originales (64.6%). Por otro lado, con ventanas de 7.5 s se logro una exactitud de 72.9%, lo que representa cuatro sujetos adicionales correctamente clasificados respecto a la condición de referencia.

Por lo tanto, debido a que tanto en 6s como en 7.5s se obtuvieron desempeños superiores respecto a la referencia, se tomo la decisión de evaluar las diferentes estrategias de adaptación propuestas con el fin de comprobar si la metodología se mantiene a lo largo de diferentes analisis, además si es dependiente de la duración de la entrada.

C. Tamaño de ventana de 6 segundos

En esta sección se comparan todas las estrategias de ajuste evaluadas con ventanas de 6 s frente a su propio *baseline* (wav2vec 2.0 como extractor de caracteristicas). *En la Tabla IV p*odemos observar que el modelo original congelado (*baseline)* alcanza una exactitud de 66.7%.

Cuando comparamos el descongelamiento parcial con este punto inicial, ambas variantes la superan, aunque con mejoras modestas, concretamente 6.3 puntos porcentuales (tres sujetos) en la configuración progresiva y 2.1 puntos (un sujeto) en la inteligente. Este resultado es coherente con el sobreajuste temprano observado durante su entrenamiento, esta estrategia modifica directamente millones de pesos de un modelo preentrenado con miles de horas de habla, y con solo 108 sujetos de entrenamiento el ajuste tiende a memorizar los datos disponibles en lugar de aprovechar el conocimiento previamente adquirido. Además, en esta familia la variante progresiva supera a la inteligente, a diferencia de lo que ocurre con los métodos basados en módulos adicionales.

TABLA IV: RESULTADO DE EXPERIMENTOS PARA VENTANAS DE 6 SEGUNDOS

Escenario	Tipo	Configuración	Exactitud	Sensibilidad	Especificidad	F1-Score
Baseline	—	—	66.7	58.3	75	63.6
Descongelamiento	Progresivo	N=4	72.9	66.7	79.2	71.1
	Inteligente	B	68.8	79.2	58.3	71.7
Bottleneck	Estandar	Pf	64.6	79.2	50.0	69.1
	Progresivo	Ho, N=4	77.1	79.2	75.0	77.6
	Inteligente	Ho, B	83.3	83.3	83.3	83.3
LoRA	Estandar	k	62.5	62.5	62.5	62.5
	Progresivo	v,N =1	77.1	70.8	83.3	75.6
	Inteligente	v+o, C	79.2	75	83.3	78.3

Nota: Progresivo: ajuste en las N últimas capas; Inteligente: ajuste en un bloque funcional (B: capas 8–9; C: capas 10–12). Pf/Ho: adaptador tipo Pfeiffer/Houlsby; k:*key*, v: *value*, o: *output*, Todas las metricas se expresan en porcentaje (%)

Al analizar los adaptadores *bottleneck,* estos alcanzan el mejor desempeño de todo el estudio con la estrategia inteligente, con una exactitud de 83.3 %, 16.7 puntos porcentuales (ocho sujetos) por encima del *baseline*, seguida de la progresiva (77.1 %), mientras que la estándar queda por debajo de este mismo punto de referencia (64.6 %). Que la estrategia inteligente supere

a las demás tiene sentido, ya que concentra la adaptación en el bloque semántico (capas 8–9), donde se codifica la información léxica y de significado que, como se describió anteriormente, se deteriora en el habla de las personas con Alzheimer, mientras que la estándar distribuye los módulos en capas fijas a lo largo de toda la red, incluidas capas inferiores cuyo contenido es principalmente acústico.

Por otro lado, en LoRA la estrategia inteligente vuelve a obtener el mejor resultado, con una exactitud de 79.2 %, 12.5 puntos porcentuales (seis sujetos) por encima del *baseline*, seguida de la progresiva (77.1 %), mientras que la estándar nuevamente queda por debajo de la referencia (62.5 %). Esto confirma que, en los métodos basados en módulos adicionales, restringir la adaptación a bloques definidos según su contenido representacional es más efectivo que distribuirla de forma genérica a lo largo de la red.

La Fig 10, muestra la exactitud de los diferentes experimentos realizados sobre ventanas de 6 seg permitiendonos comparar visualemente tanto estrategias como métodos usados. De manera general, podemos observar que tanto Bottleneck como Lora superan tanto en el tipo inteligente como en el tipo progresivo los resultados obtenidos utilizando wav2vec 2.0 unicamente como extractor de caracteristicas.

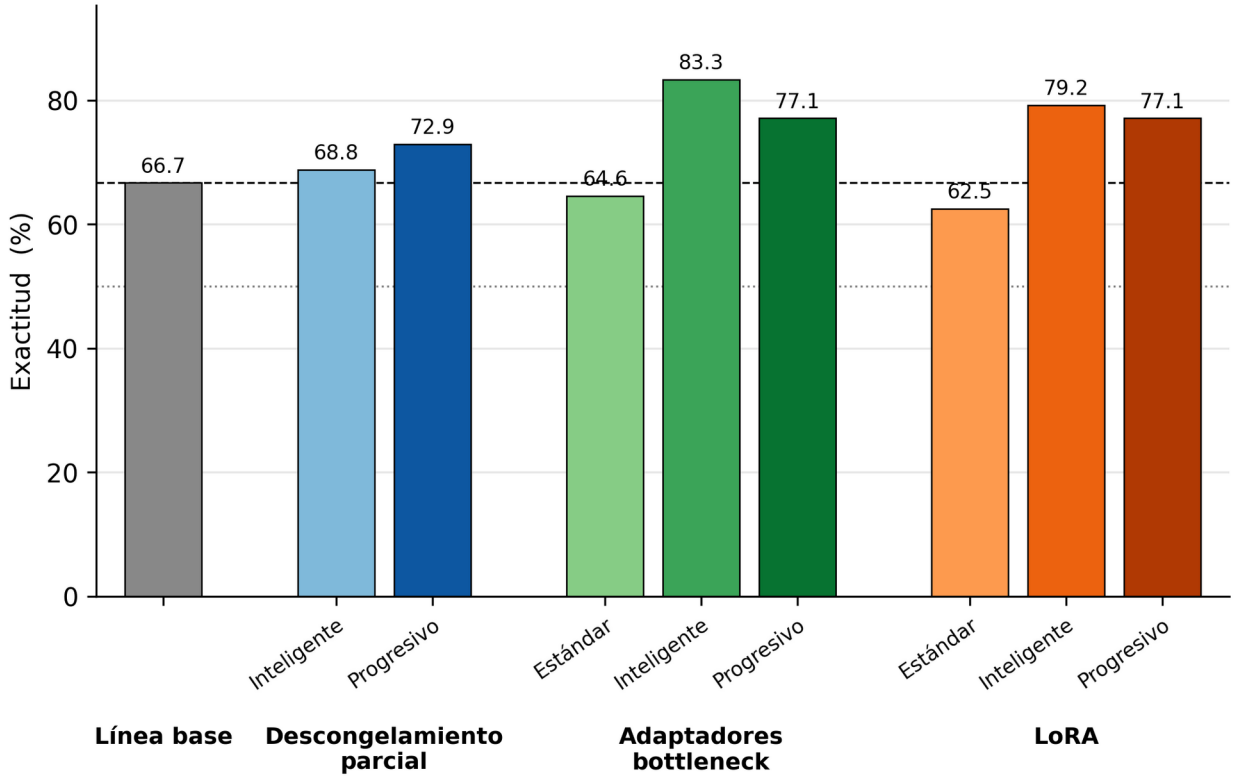


Fig. 10: Exactitud obtenida por cada configuración

Las curvas ROC (Fig. 11a) muestran que la ventaja del adaptador bottleneck inteligente no depende del umbral de decisión. Su curva se ubica por encima del *baseline* en prácticamente todo el rango. Con una tasa de falsos positivos cercana al 17 %, detecta alrededor del 83 % de los pacientes con AD, mientras que el modelo completamente congelado con la misma tasa, detecta aproximadamente el 42 %. LoRA inteligente sigue un comportamiento similar (AUC de 0.835). Ambos modelos mejoran la separación entre clases en toda la población, y no solo en torno al punto de operación elegido.

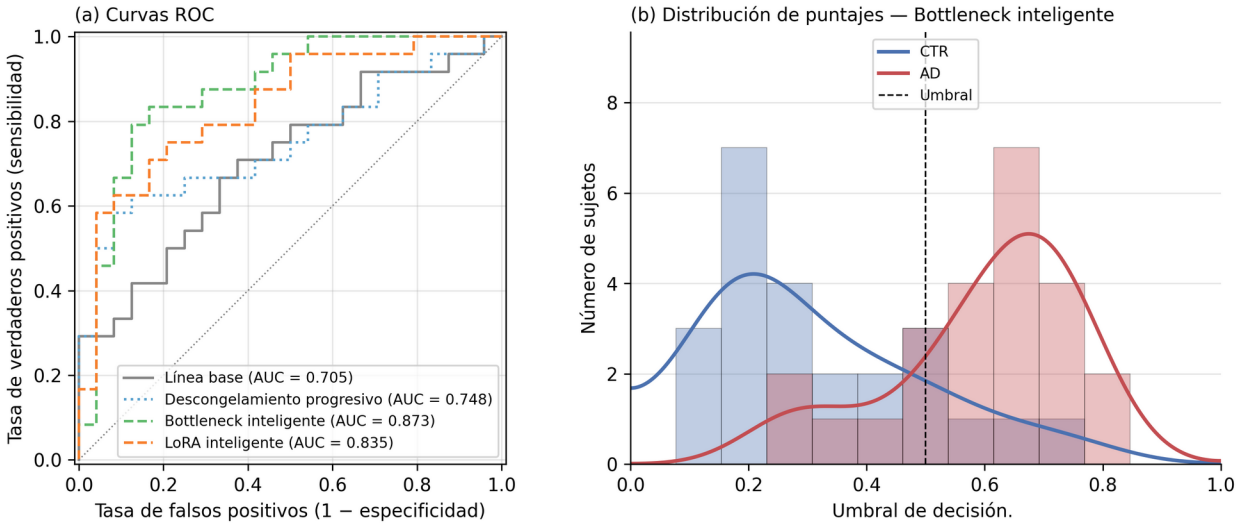


Fig. 11: Curvas ROC y diagramas de distribución para el mejor modelo de 6 seg

El descongelamiento progresivo muestra un patrón distinto. Su curva supera a la del *baseline* solo cuando la tasa de falsos positivos es baja, por debajo del 10 % aproximadamente, donde detecta cerca del 60 % de los pacientes. A partir de una tasa cercana al 35 %, ambas curvas se superponen. Esto indica que el descongelamiento separa mejor a los sujetos con puntajes más extremos, los casos más evidentes de cada clase, pero no mejora la discriminación de los sujetos con puntajes intermedios, que son los que determinan el desempeño en el resto de la curva. Su AUC (0.748), apenas superior (0.705), refleja esta mejora parcial.

La distribución de puntajes del adaptador bottleneck inteligente (Fig. 11b) muestra que el modelo separa a la mayoría de los sujetos con un margen amplio respecto al umbral. De los 24 controles, 14 reciben puntajes inferiores a 0.31, y de los 24 pacientes con AD, 17 se ubican por encima de 0.54, con una concentración marcada entre 0.6 y 0.77. Para estos sujetos, la decisión del modelo no depende del valor exacto del umbral. Los errores del modelo se reparten de forma equilibrada entre ambas clases, el número de pacientes con puntajes bajos es similar al de controles con puntajes altos. Esto explica que la sensibilidad y la especificidad sean idénticas (83.3 %) y confirma que el modelo no favorece a ninguna de las dos clases.

Este resultado puede interpretarse a partir del funcionamiento del adaptador. Cada módulo bottleneck recibe la representación producida por una subcapa, la proyecta a un espacio de menor dimensión, la transforma y la devuelve a la dimensión original, sumándola a la representación de entrada mediante una conexión residual. En la práctica, el adaptador actúa como una traducción entre la representación de una capa y la que recibe la siguiente, conserva la información original y agrega solo la corrección necesaria para la tarea. Ubicada en el bloque semántico, permite reorientar hacia la clasificación de AD las representaciones del modelo, sin modificar los pesos que las generan.

El ultimo punto del análisis relaciona el desempeño de cada configuración con el número de parámetros del modelo base que modifica. La Tabla V presenta el número de parámetros entrenables de cada configuración, su proporción respecto al total del modelo y la reducción frente al ajuste fino completo.

Si la cantidad de parámetros entrenables determinara el desempeño, la exactitud debería aumentar a medida que crece el número de parámetros. Los resultados muetran que las dos variantes de descongelamiento parcial, que entrenan entre el 15% y el 30% del modelo (entre 14 y 28 millones de parámetros), se ubican entre las configuraciones de menor exactitud (68.8% y 72.9%), mientras que el mejor resultado (83.3%) lo obtiene el adaptador *bottleneck* inteligente, que entrena apenas el 0.166 % del modelo, 602 veces menos parámetros que el modelo completo y unas 90 veces menos que el descongelamiento parcial en el mismo bloque de capas.

TABLA V: PARAMETROS ENTRENABLES POR CONFIGURACIÓN EN VENTANAS DE 6 SEGUNDOS

Escenario	Tipo	# Capas modificadas	Parámetros entrenables	% del modelo ajustaddo	Exactitud
Baseline	-	0	0	0	66.7
Descongelamiento	Progresivo	4	28,351,488	30.04	72.9
	Inteligente	2	14,175,744	15.02	68.8
Bottleneck	Estandar	4	304,320	0.322	64.6
	Progresivo	4	313,536	0.332	77.1
	Inteligente	2	156,768	0.166	83.3
LoRA	Estandar	4	24,576	0.026	62.5
	Progresivo	1	24,576	0.026	77.1
	Inteligente	3	147,456	0.156	79.2

La evidencia más directa surge al comparar configuraciones con un presupuesto de parámetros equivalente. En LoRA, las variantes estándar y progresiva entrenan exactamente 24,576 parámetros, y la exactitud pasa de 62.5% a 77.1%. En los adaptadores *bottleneck*, las variantes estándar y progresiva entrenan un número similar de parámetros (304,320 y 313,536) sobre el mismo número de capas (cuatro), y la exactitud pasa de 64.6% a 77.1%. En ambos casos, la diferencia no proviene de la capacidad de adaptación, sino principalmente de dónde se aplica, las variantes estándar reparten el ajuste entre capas inferiores, intermedias y superiores, mientras que las progresivas lo concentran en las últimas capas del modelo. Por lo tanto, en condiciones de datos limitados, reducir el número de parámetros entrenables no basta; la eficiencia se obtiene cuando la adaptación se ubica en las capas cuyo contenido es relevante para la tarea.

C. Resultados con ventana de 7.5 segundos

En esta sección se comparan los mismos experimentos de la sección anterior, ahora evaluados con ventanas de 7.5 s frente a su propio *baseline*. Con esta duración, el modelo

congelado utilizado unicamente como extractor de caracteristicas alcanza una exactitud de 72.9%, un punto de partida más exigente que el de la ventana de 6 s. Cuando se compara el descongelamiento parcial con esta referencia, se observa el cambio más notable respecto a la ventana anterior, ambas variantes quedan por debajo de la línea base, con 4.2 puntos porcentuales menos (dos sujetos) en la configuración progresiva y 2.1 puntos menos (un sujeto) en la inteligente. Una posible explicación es que las ventanas más largas producen menos ejemplos de entrenamiento por sujeto, lo que agrava el riesgo de sobreajuste en una estrategia que modifica directamente millones de pesos de un modelo preentrenado. Además, a diferencia de lo observado con 6 s, en esta familia la variante inteligente supera a la progresiva.

TABLA VI: RESULTADO DE EXPERIMENTOS PARA VENTANAS DE 7.5 SEGUNDOS

Escenario	**Tipo**	**Configuración**	**Exactitud**	**Sensibilidad**	**Especificidad**	**F1-Score**
Baseline	—	—	72.9	66.7	79.2	71.1
Descongelamiento	Progresivo	N = 6	68.8	66.7	70.8	68.1
	Inteligente	C	70.8	54.2	87.5	65.0
Bottleneck	Estandar	Pf	60.4	62.5	58.3	61.2
	Progresivo	Pf, N = 9	77.1	62.5	91.7	73.2
	Inteligente	Ho, A	75.0	75.0	75.0	75.0
LoRA	Estandar	k	72.9	54.2	91.7	66.7
	Progresivo	q y k, N = 5	77.1	66.7	87.5	74.4
	Inteligente	v y o, B	77.1	79.2	75.0	77.6

Nota: Progresivo: ajuste en las N últimas capas; Inteligente: ajuste en un bloque funcional (A: capas 6-7; B: capas 8–9; C: capas 10–12). Pf/Ho: adaptador tipo Pfeiffer/Houlsby; k:key, v: value, o: output, Todas las metricas se expresan en porcentaje (%)

De los resultados registrados en la Tabla VI, podemos analizar los adaptadores *bottleneck*, las estrategias progresiva e inteligente vuelven a superar el punto de referencia, aunque con márgenes menores que en la ventana de 6 s: la progresiva alcanza 77.1 %, 4.2 puntos porcentuales (dos sujetos) por encima de la referencia, y la inteligente 75.0 %, 2.1 puntos (un

sujeto) por encima. En este caso, la progresiva obtiene la mayor exactitud, pero a costa de un comportamiento desbalanceado: su especificidad llega a 91.7 %, mientras que su sensibilidad (62.5 %) queda por debajo. La inteligente, en cambio, mantiene un desempeño perfectamente equilibrado entre ambas clases (75.0 % de sensibilidad y especificidad) y obtiene el mayor F1 de esta familia (75.0 %). La estrategia estándar, por su parte, vuelve a quedar rezagada y obtiene la menor exactitud de todos los experimentos (60.4 %), 12.5 puntos porcentuales (seis sujetos) por debajo de la referencia.

Por otro lado, en LoRA las estrategias progresiva e inteligente alcanzan la misma exactitud (77.1 %), 4.2 puntos porcentuales (dos sujetos) por encima del *baseline*, pero la inteligente lo logra con una sensibilidad de 79.2 %, la más alta de todos los experimentos con esta duración de ventana, y un F1 de 77.6 %, también el mayor. La estrategia estándar, en cambio, iguala el modelo congelado (72.9 %), pero esta cifra oculta un peor comportamiento: su sensibilidad cae a 54.2 %, es decir, deja de detectar a casi la mitad de los pacientes, y su exactitud se sostiene únicamente porque clasifica a la mayoría de los sujetos como controles. Como en la ventana de 6 s, en ambos métodos basados en módulos adicionales la estrategia estándar no logra mejorar la referencia, mientras que restringir la adaptación a un bloque funcional ofrece el desempeño más equilibrado entre clases.

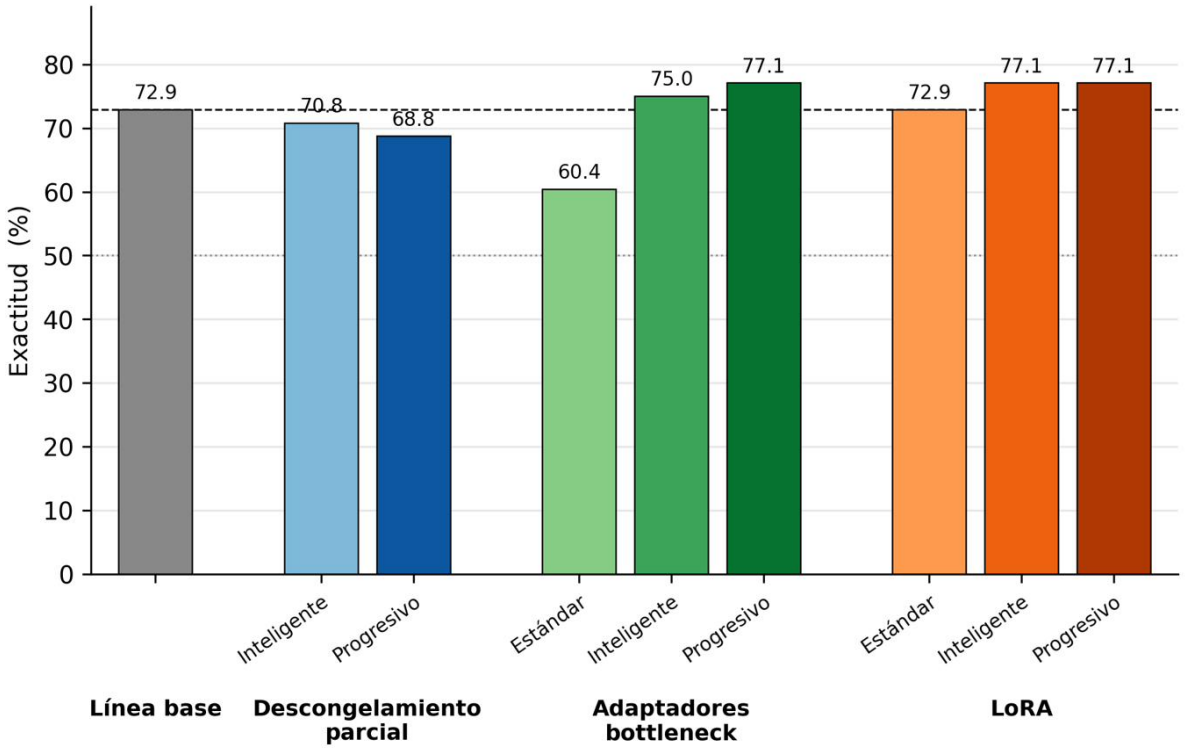


Fig. 12. Exactitud de modelos para ventanas de 7.5 segundos

La Fig. 12 resume la exactitud de todos los experimentos realizados con ventanas de 7.5 s. Al igual que en la ventana de 6 s, se observa que nuevamente los adaptadores *bottleneck* y LoRA, en sus estrategias progresiva e inteligente, superan el *baseline*, mientras que el descongelamiento parcial y las configuraciones estándar no logran mejorarla. Que este patrón se repita en ambas duraciones de ventana, aun cuando con 7.5 s la referencia es más exigente y los márgenes de mejora son menores , indica que las estrategias propuestas son una alternativa viable para adaptar modelos preentrenados de gran tamaño en escenarios con pocos datos, y que su ventaja se mantiene frente a cambios en la granularidad de la señal de entrada.

La Fig. 13a muestra las curvas ROC de los mejores resultados obtenidos para las ventana de 7.5 s. Tres configuraciones comparten la mayor exactitud (77.1 %), el adaptador bottleneck progresivo, LoRA progresivo y LoRA inteligente. Entre ellas se destaca LoRA inteligente, con matrices v y o en el bloque semántico (capas 8 y 9), por obtener el mayor AUC (0.818) y el comportamiento más balanceado entre clases. La forma en que cada modelo supera al *baseline* es

distinta. El adaptador bottleneck progresivo alcanza una especificidad de 91.7 %, pero su sensibilidad (62.5 %) es inferior a la del *baseline*, su ganancia proviene de reconocer mejor a los controles, no de detectar más pacientes. LoRA inteligente hace lo contrario: eleva la sensibilidad de 66.7 % a 79.2 % a cambio de una leve reducción en la especificidad (de 79.2 % a 75.0 %), lo que lo convierte en el modelo más útil para identificar sujetos con AD.

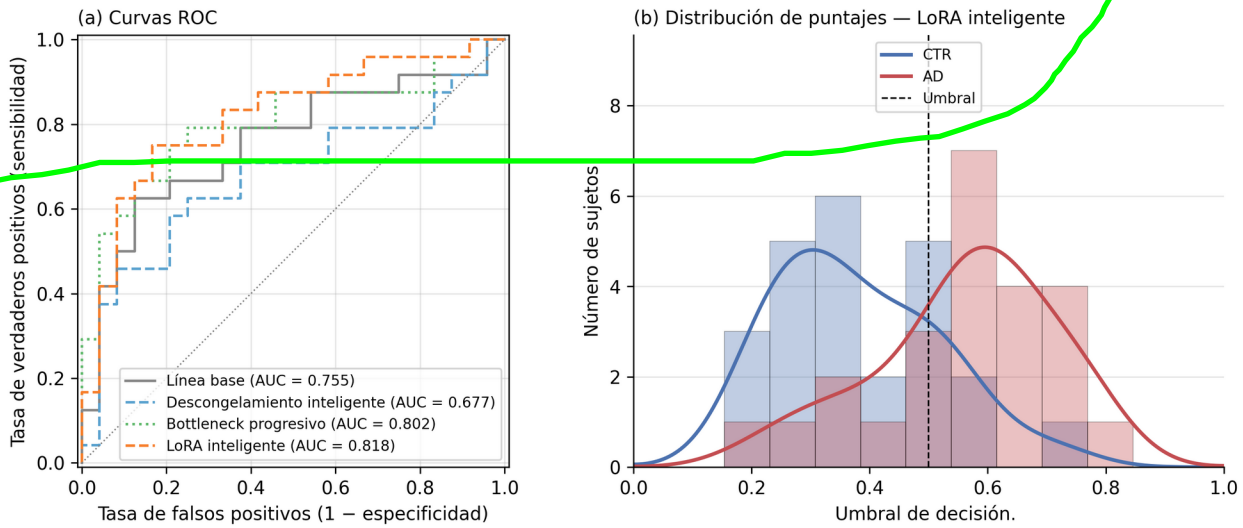


Fig. 13: Curvas ROC y diagramas de distribución para el mejor modelo de 7.5 segundos

Las curvas muestran que, con esta ventana, las diferencias entre modelos son más estrechas que con 6 s. LoRA inteligente y el adaptador bottleneck progresivo se separan del baseline principalmente en la región intermedia de tasas de falsos positivos, con una tasa cercana al 17 %, LoRA inteligente detecta alrededor del 75 % de los pacientes, frente a aproximadamente el 62 % del modelo congelado. El descongelamiento inteligente, en cambio, queda por debajo del baseline en la mayor parte del rango (AUC de 0.677). A diferencia de lo observado en 6 s, su menor exactitud no se debe al umbral de decisión, sino a una capacidad de discriminación realmente inferior al del modelo congelado. Por otro lado, la distribución de puntajes de LoRA inteligente mostrada en la Fig. 13B permite observar a los controles concentrados alrededor de 0.3 y a los pacientes alrededor de 0.6. Los dos picos están más cercanos que en el mejor modelo de 6 s, y la región de superposición es más amplia, entre 0.15 y 0.55 aproximadamente. Esto

explica la mayor cantidad de errores de clasificación y el menor AUC respecto al adaptador bottleneck inteligente de la ventana de 6 s.

La Tabla VII presenta el número de parámetros entrenables de cada configuración con ventana de 7.5 s, y la Fig. 14 los relaciona con la exactitud obtenida. Con esta duración de ventana, la relación entre parámetros y desempeño es incluso más clara que con 6 s. Las dos configuraciones con más parámetros (descongelamiento parcial) son las únicas familias que no logran superar al baseline. En el extremo opuesto, LoRA inteligente alcanza el mejor desempeño (77.1 %) entrenando solo 24,576 parámetros, que corresponde al el 0.026 % de los parámetros que conforman el modelo, el mismo resultado que el adaptador bottleneck progresivo con 55 veces más parámetros.

Las configuraciones estándar confirman el patrón. En LoRA, las configuraciones estándar e inteligente entrenan exactamente el mismo número de parámetros, y la exactitud pasa de 72.9 % a 77.1 %, con un AUC que sube de 0.668 a 0.818. En los adaptadores bottleneck el contraste es mayor: la configuración estándar, con 599,424 parámetros, obtiene 60.4 %, mientras que la inteligente, con la mitad de parámetros, alcanza 75.0 %. Es decir, ubicar de manera estrategia la adaptación con menos parámetros supera a ubicarla mal con más.

En conjunto, el análisis con 7.5 s es congruente con lo observado en 6 s. En ambas ventanas, el descongelamiento parcial, que es la estrategia con mayor número de parámetros entrenables, obtiene mejoras modestas o queda por debajo de la línea base, mientras que los mejores resultados se alcanzan entrenando una fracción mínima del modelo: 0.166 % con el adaptador *bottleneck* inteligente en 6 s y 0.026 % con LoRA inteligente en 7.5 s. De igual forma, en ambas ventanas las comparaciones entre configuraciones con un presupuesto de parámetros igual o similar muestran diferencias de entre 4.2 y 14.6 puntos porcentuales, que solo pueden atribuirse a dónde se ubica la adaptación. Esto confirma que, en condiciones de datos limitados, el número de parámetros entrenables no determina el desempeño: lo que marca la diferencia es

adaptar las capas adecuadas, de modo que una adaptación bien ubicada con pocos parámetros supera de forma consistente a una mal ubicada con muchos más.

TABLA VII: PARAMETROS ENTRENABLES POR CONFIGURACIÓN EN VENTANAS DE 6 SEGUNDOS

Escenario	**Tipo**	**# Capas modificadas**	**Parámetros entrenables**	**% del modelo ajustaddo**	**Exactitud**
Baseline	-	0	0	0	72.9
Descongelamiento	Progresivo	6	42,527,232	45.06	68.8
	Inteligente	3	21,263,616	22.53	70.8
Bottleneck	Estandar	4	599,424	0.635	60.4
	Progresivo	9	1,348,704	1.429	77.1
	Inteligente	2	304,320	0.322	75.0
LoRA	Estandar	4	24,576	0.026	72.9
	Progresivo	5	86,016	0.091	77.1
	Inteligente	2	24,576	0.026	77.1

VI. DISCUSIÓN

Los resultados de ambas duraciones de ventana muestran un patrón consistente: en 6 s y nuevamente en 7.5 s, los adaptadores *bottleneck* y LoRA superan el *baseline* con las estrategias progresiva e inteligente, mientras que el descongelamiento parcial y las configuraciones estándar no logran hacerlo de forma sistemática. Esto indica que adaptar el modelo mediante módulos adicionales ubicados en capas seleccionadas es una alternativa viable para aprovechar modelos preentrenados de gran tamaño con pocos datos, y que su ventaja no depende de una única duración de ventana. Sin embargo, la magnitud de la mejora fue ~~considerablemente~~ mayor con 6 s (hasta 16.7 puntos porcentuales) que con 7.5 s (hasta 4.2 puntos). Una posible explicación es que las ventanas más cortas generan un mayor número de ejemplos de entrenamiento por sujeto, lo que favorece a los métodos que deben aprender nuevos parámetros a partir de un corpus pequeño. A esto se suma que, con 7.5 s, el modelo congelado ya parte de una referencia más alta (72.9 % frente a 66.7 %), posiblemente porque al promediar las representaciones sobre una ventana más larga obtiene una descripción más estable de cada fragmento, lo que reduce el margen disponible para la adaptación.

En cuanto a las estrategias, la inteligente ofrece el mejor balance entre desempeño y número de capas adaptadas. Con ella se obtuvo el mejor resultado de todo el estudio (83.3 % con adaptadores *bottleneck* en 6 s) y, con 7.5 s, las configuraciones inteligentes de ambos métodos presentaron el desempeño más equilibrado entre clases, con la mayor sensibilidad y el mayor F1 de su familia, mientras que las progresivas alcanzaron una exactitud igual o ligeramente superior a costa de detectar menos pacientes y de adaptar más capas. El contraste con la estrategia estándar, que distribuye el ajuste a lo largo de toda la red y fue la única que no superó a la línea base en ninguna de las dos ventanas, refuerza esta idea, no basta con añadir módulos de adaptación, sino que es necesario ubicarlos en las capas cuyo contenido resulta relevante para la tarea. En consecuencia, adaptar un modelo preentrenado de forma eficiente requiere un conocimiento mínimo de cómo se organiza la información en su interior, como el que proporcionan los análisis por capas, para decidir dónde intervenir en lugar de hacerlo de manera genérica o exhaustiva.

VI. CONCLUSIONES Y RECOMENDACIONES

En este trabajo se evaluó la adaptación de wav2vec 2.0 para la detección automática del Alzheimer a partir del habla espontánea, en un escenario con pocos datos. Con ventanas de 6 s, el mejor resultado se obtuvo al insertar adaptadores *bottleneck* tipo Houlsby en el bloque semántico (capas 8–9), con una exactitud de 83.3 %, 16.7 puntos porcentuales por encima del modelo empleado únicamente como extractor de características. Con ventanas de 7.5 s, la mayor exactitud fue de 77.1 %, 4.2 puntos porcentuales por encima del *baseline*, y la obtuvieron tanto los adaptadores *bottleneck* y LoRA con la estrategia progresiva como LoRA con la estrategia inteligente en el bloque semántico, siendo esta última la de desempeño más equilibrado entre clases.

Los resultados de este trabajo permiten afirmar que el ajuste eficiente de parámetros es una alternativa viable y preferible al ajuste de pesos del modelo para detectar AD a partir del habla cuando los datos son escasos. Sin embargo, esta ventaja no es intuitiva, depende de decisiones de diseño que suelen tratarse como secundarias. La primera de estas decisiones es la duración del audio que recibe el modelo. wav2vec 2.0 construye la representación de cada instante a partir del contexto que lo rodea, y fue preentrenado con fragmentos largos, cuando se le entregan fragmentos cortos, pierde el contexto del que dependen sus representaciones. Antes de ajustar el modelo es necesario garantizar que cada fragmento le ofrezca un contexto suficiente al de su preentrenamiento.

La segunda, es la ubicación donde se realiza la adaptación. El número de parámetros entrenables no explica por sí solo el desempeño de los modelos. Se obtuvieron buenos resultados con configuraciones cuyo número de parámetros difiere en varios órdenes de magnitud respecto al numero total de parametros del modelo original, y resultados deficientes con cantidades idénticos a los de configuraciones exitosas. Lo que tienen en común las configuraciones que superan el modelo congelado no es cuántos parámetros entrenan, sino dónde lo hacen, y una adaptación mal ubicada puede resultar incluso peor que no adaptar el modelo. La pregunta relevante al aplicar estos métodos es, en qué capas adaptar el modelo, y el análisis del contenido representacional de cada capa demostró ser una herramienta útil para responderla.

Por último, los resultados sugieren que el conocimiento que wav2vec 2.0 adquiere durante su preentrenamiento es transferible a esta tarea. La forma en que el modelo relaciona los distintos instantes de la señal a partir del contexto parece adecuada tal como está; lo que requiere ajuste es el contenido que el modelo extrae de ella. Los dos métodos con mejor desempeño apuntan en esa dirección. En LoRA, la adaptación más consistente fue la de las matrices asociadas al contenido de la atención. Los adaptadores bottleneck, por su parte, actúan como una traducción entre las representaciones de una capa y la siguiente, agregando una corrección sobre la información que el modelo ya produce. En ambos casos, la adaptación efectiva actúa sobre el contenido de las representaciones y no sobre la forma en que el modelo relaciona los distintos instantes de la señal.

REFERENCIAS

[1] "Global action plan on the public health response to dementia 2017 - 2025". Consultado: el 25 de septiembre de 2026. [En línea]. Disponible en: https://www.who.int/publications/i/item/global-action-plan-on-the-public-health-response-to-dementia-2017---2025

[2] "La epidemia de demencia avanza silenciosa con el envejecimiento en Colombia". Consultado: el 25 de septiembre de 2026. [En línea]. Disponible en: https://www.periodico.unal.edu.co/articulos/la-epidemia-de-demencia-avanza-silenciosa-con-el-envejecimiento-en-colombia

[3] A. Guerrero-Barragán y D. I. Lucumi-Cuesta, "Stakeholder perceptions of dementia in Colombia: a qualitative study", *BMC Public Health*, vol. 25, núm. 1, p. 959, mar. 2025, doi: 10.1186/s12889-025-22166-9.

[4] A. Giudicessi *et al.*, "Cognitive Outcomes in Autosomal-Dominant Alzheimer's Disease: A Comprehensive Review from a Colombian Kindred with the Presenilin-1 E280A Mutation", *Journal of Alzheimer's Disease*, vol. 101, núm. 2, pp. 397–415, sep. 2024, doi: 10.3233/JAD-240360.

[5] I. Vigo, L. Coelho, y S. Reis, "Speech- and Language-Based Classification of Alzheimer's Disease: A Systematic Review", *Bioengineering*, vol. 9, núm. 1, p. 27, ene. 2022, doi: 10.3390/bioengineering9010027.

[6] K. C. Fraser, J. A. Meltzer, y F. Rudzicz, "Linguistic Features Identify Alzheimer's Disease in Narrative Speech", *JAD*, vol. 49, núm. 2, pp. 407–422, nov. 2015, doi: 10.3233/JAD-150520.

[7] A. D. Assfaw, S. E. Schindler, y J. C. Morris, "Advances in blood biomarkers for Alzheimer disease (AD): A review", *The Kaohsiung J of Med Scie*, vol. 40, núm. 8, pp. 692–698, ago. 2024, doi: 10.1002/kjm2.12870.

[8] A. Baevski, H. Zhou, A. Mohamed, y M. Auli, "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations", el 22 de octubre de 2020, *arXiv*: arXiv:2006.11477. doi: 10.48550/arXiv.2006.11477.

[9] S. Luz, F. Haider, S. de la Fuente, D. Fromm, y B. MacWhinney, "Alzheimer's Dementia Recognition through Spontaneous Speech: The ADReSS Challenge", el 5 de agosto de 2020, *arXiv*: arXiv:2004.06833. doi: 10.48550/arXiv.2004.06833.

[10] F. Haider, S. De La Fuente, y S. Luz, "An Assessment of Paralinguistic Acoustic Features for Detection of Alzheimer's Dementia in Spontaneous Speech", *IEEE J. Sel. Top. Signal Process.*, vol. 14, núm. 2, pp. 272–281, feb. 2020, doi: 10.1109/JSTSP.2019.2955022.

[11] C. D. Rios-Urrego, J. C. Vásquez-Correa, J. R. Orozco-Arroyave, y E. Nöth, "Transfer Learning to Detect Parkinson's Disease from Speech In Different Languages Using Convolutional Neural Networks with Layer Freezing", en *Text, Speech, and Dialogue*, vol. 12284, P. Sojka, I. Kopeček, K. Pala, y A. Horák, Eds., en Lecture Notes in Computer Science, vol. 12284. , Cham: Springer International Publishing, 2020, pp. 331–339. doi: 10.1007/978-3-030-58323-1_36.

[12] A. Pasad, J.-C. Chou, y K. Livescu, "Layer-wise Analysis of a Self-supervised Speech Representation Model", el 3 de diciembre de 2022, *arXiv*: arXiv:2107.04734. doi: 10.48550/arXiv.2107.04734.

[13] V. Berisha, C. Krantsevich, G. Stegmann, S. Hahn, y J. Liss, “Are reported accuracies in the clinical speech machine learning literature overoptimistic?”, en *Interspeech 2022*, ISCA, sep. 2022, pp. 2453–2457. doi: 10.21437/Interspeech.2022-691.

[14] Y. Xiong, V. Berisha, J. Liss, y C. Chakrabarti, “Mitigating Overfitting During Speech Foundation Model Fine-tuning: Applications to Dysarthric Speech Detection”, en *Interspeech 2025*, ISCA, ago. 2025, pp. 2133–2137. doi: 10.21437/Interspeech.2025-567.

[15] J. Howard y S. Ruder, “Universal Language Model Fine-tuning for Text Classification”.

[16] N. Houlsby *et al.*, “Parameter-Efficient Transfer Learning for NLP”, el 13 de junio de 2019, *arXiv*: arXiv:1902.00751. doi: 10.48550/arXiv.1902.00751.

[17] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models”, el 16 de octubre de 2021, *arXiv*: arXiv:2106.09685. doi: 10.48550/arXiv.2106.09685.

[18] T. Purohit, B. Ruvolo, J. R. Orozco-Arroyave, y M. Magimai.-Doss, “Automatic Parkinson’s disease detection from speech: Layer selection vs adaptation of foundation models”, en *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Hyderabad, India: IEEE, abr. 2025, pp. 1–5. doi: 10.1109/ICASSP49660.2025.10890852.

[19] D. S. Park *et al.*, “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition”, en *Interspeech 2019*, ISCA, sep. 2019, pp. 2613–2617. doi: 10.21437/Interspeech.2019-2680.

[20] V. Lialin, V. Deshpande, X. Yao, y A. Rumshisky, “Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning”, el 22 de noviembre de 2024, *arXiv*: arXiv:2303.15647. doi: 10.48550/arXiv.2303.15647.

[21] J. Pfeiffer, A. Kamath, A. Rücklé, K. Cho, y I. Gurevych, “AdapterFusion: Non-Destructive Task Composition for Transfer Learning”, en *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, Online: Association for Computational Linguistics, 2021, pp. 487–503. doi: 10.18653/v1/2021.eacl-main.39.

[22] G. Strang, *Linear algebra and its applications*, 4th ed. Belmont, CA: Thomson, Brooks/Cole, 2006.

[23] “The Natural History of Alzheimer’s Disease: Description of Study Cohort and Accuracy of Diagnosis”.

[24] C. Wang *et al.*, “VoxPopuli: A Large-Scale Multilingual Speech Corpus for Representation Learning, Semi-Supervised Learning and Interpretation”, en *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Online: Association for Computational Linguistics, 2021, pp. 993–1003. doi: 10.18653/v1/2021.acl-long.80.

[25] N. Elhage *et al.*, “A Mathematical Framework for Transformer Circuits”, *Transformer Circuits Thread*, 2021.

Anexo A Póster Autoarchivo en Repositorio y documentos de interés

Departamento de Ingeniería Electrónica y de Telecomunicaciones

Ajuste fino de parámetros sobre wav2vec 2.0 para la clasificación de la enfermedad de Alzheimer mediante señales de voz en condiciones de datos limitados

UNIVERSIDAD DE ANTIOQUIA
Facultad de Ingeniería

PRACTICANTE: Santiago Álvarez Zuluaga
ASESOR: Cristian David Ríos Urrego
PROGRAMA: Ingeniería de Telecomunicaciones
Semestre de la práctica: 2026-2

Introducción

La enfermedad de Alzheimer (EA) es la causa más frecuente de demencia y su detección temprana depende de evaluaciones clínicas costosas y poco accesibles. El habla espontánea es un **biomarcador no invasivo**: refleja alteraciones léxicas, semánticas y prosódicas asociadas al deterioro cognitivo.

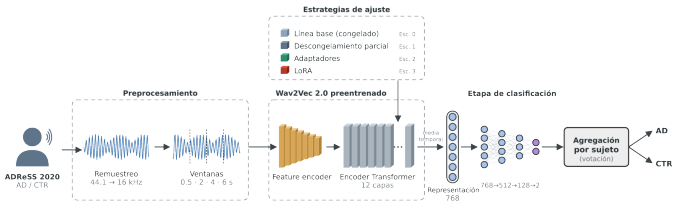
Los modelos fundacionales auto-supervisados como **wav2vec 2.0** ofrecen representaciones robustas del habla, pero ajustar todos sus parámetros con bases de datos clínicas pequeñas conduce al sobreajuste.

En este trabajo se evaluaron estrategias de **ajuste eficiente de parámetros (PEFT)**, adaptadores bottleneck y LoRA, junto con el descongelamiento parcial de capas. Cada mecanismo se aplicó en tres ubicaciones del Transformer: **estándar, progresiva e inteligente**, esta última guiada por el análisis por capas de Pasad et al. Todas se compararon con wav2vec 2.0 congelado como extractor de características (línea base).

156	≈95 M	6 s · 7.5 s
sujetos de la base de datos ADReSS	parámetros en wav2vec 2.0 base	duraciones de ventana evaluadas

Metodología

Todas las configuraciones comparten la misma cabeza de clasificación y protocolo: hiperparámetros seleccionados por exactitud en validación y evaluación única en la partición oficial de prueba (48 sujetos).



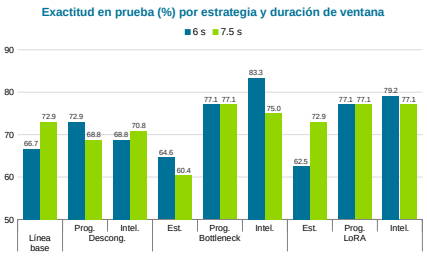
Objetivos

- Establecer una línea base de clasificación de pacientes con EA y sujetos de control con representaciones de wav2vec 2.0 sin ajuste de parámetros.
- Identificar estrategias de ajuste de parámetros aplicables a la arquitectura de wav2vec 2.0 en escenarios con datos limitados.
- Implementar diferentes estrategias de ajuste sobre wav2vec 2.0 para adaptarlo a la clasificación entre pacientes con EA y controles.
- Evaluar y comparar el desempeño, los tiempos de entrenamiento e inferencia y los parámetros entrenables entre el modelo base y las estrategias implementadas.

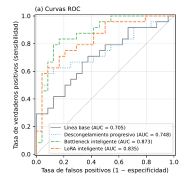
Resultados

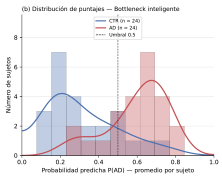
Con ventanas de 6 s, el adaptador **bottleneck inteligente** (Houlsby, bloque B) logró el mejor desempeño: **83.3 %** de exactitud, sensibilidad y especificidad (AUC = 0.873), frente a 66.7 % de la línea base.

Con 7.5 s la línea base fue más exigente (72.9 %), pero **bottleneck y LoRA progresivos e inteligentes** volvieron a superarla (75.0–77.1 %). Las variantes estándar y el descongelamiento parcial no la mejoraron de forma consistente.



83.3 %	exactitud del bottleneck inteligente con 6 s
+16.7 p. p.	sobre la línea base (8 sujetos más)
0.166 %	de parámetros entrenados (156,768)





Conclusiones

- **Bottleneck y LoRA**, con estrategias progresiva e inteligente, superaron a la línea base con 6 s y 7.5 s: PEFT es viable para adaptar wav2vec 2.0 con pocos datos.
- **La ubicación importa más que el número de parámetros**: con el mismo presupuesto (24,576 parámetros), LoRA pasó de 62.5 % en la variante estándar a 77.1 % en la progresiva (6 s).
- **El descongelamiento parcial**, con millones de parámetros, sobreajustó desde las primeras épocas y quedó por debajo de la línea base con 7.5 s.
- **Adaptar el bloque semántico** (capas 8–9) con el 0.166 % de los parámetros dio el mejor resultado, en línea con el deterioro léxico-semántico del habla en la EA.

DATOS DE CONTACTO DEL AUTOR:
+57 3XX XXX XXXX
usuario@udea.edu.co
linkedin.com/in/usuario